

# **High-Performance Network Intrusion Detection Engine Software**

**A PROJECT REPORT**

*Submitted by,*

|                             |          |                     |
|-----------------------------|----------|---------------------|
| <b>Urja Savalekar</b>       | <b>-</b> | <b>20211CCS0044</b> |
| <b>Adithya. R</b>           | <b>-</b> | <b>20211CCS0047</b> |
| <b>K Md Hizqueel Danish</b> | <b>-</b> | <b>20211CCS0076</b> |
| <b>Prince Gupta</b>         | <b>-</b> | <b>20211CCS0096</b> |

*Under the guidance of,*

**Dr. SHARMASTH VALI. Y**

*in partial fulfillment for the award of the degree of*

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND ENGINEERING, CYBER SECURITY**

**At**



**PRESIDENCY UNIVERSITY**

**BENGALURU**

**JANUARY 2025**

# **PRESIDENCY UNIVERSITY**

## **SCHOOL OF COMPUTER SCIENCE ENGINEERING**

### **CERTIFICATE**

This is to certify that the Project report “**HIGH PERFORMANCE NETWORK INTRUSION DETECTION ENGINE SOFTWARE**” being submitted by “Urja Savalekar, Adithya R, K Md Hizqueel Danish, Prince Gupta” bearing roll number(s) “20211CCS0044, 20211CCS0047, 20211CCS0076, 20211CCS0096” in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a bonafide work carried out under my supervision.

**Dr SHARMASTH VALI Y**

Associate Professor  
School of CSE&IS  
Presidency University

**Dr. ANAND RAJ**

Professor & HoD  
School of CSE&IS  
Presidency University

**Dr. L. SHAKKEERA**

Associate Dean  
School of CSE  
Presidency University

**Dr. MYDHILI NAIR**

Associate Dean  
School of CSE  
Presidency University

**Dr. SAMEERUDDIN KHAN**

Pro-Vc School of Engineering  
Dean -School of CSE&IS  
Presidency University

# **PRESIDENCY UNIVERSITY**

## **SCHOOL OF COMPUTER SCIENCE ENGINEERING**

### **DECLARATION**

We hereby declare that the work, which is being presented in the project report entitled **HIGH PERFORMANCE NETWORK INTRUSION DETECTION ENGINE SOFTWARE** in partial fulfillment for the award of Degree of **Bachelor of Technology** in **Computer Science and Engineering**, is a record of our own investigations carried under the guidance of **Dr. SHARMASTH VALI Y, ASSISTANT PROFESSOR, School of Computer Science Engineering & Information Science, Presidency University, Bengaluru.**

We have not submitted the matter presented in this report anywhere for the award of any other Degree.

| <b>NAME</b>                 | <b>ROLL NUMBER</b>  | <b>SIGNATURE</b> |
|-----------------------------|---------------------|------------------|
| <b>Urja Savalekar</b>       | <b>20211CCS0044</b> |                  |
| <b>Adithya. R</b>           | <b>20211CCS0047</b> |                  |
| <b>K Md Hizqueel Danish</b> | <b>20211CCS0076</b> |                  |
| <b>Prince Gupta</b>         | <b>20211CCS0096</b> |                  |

## **ABSTRACT**

This project aims to design and implement a robust Network Intrusion Detection System (NIDS) that provides real-time network traffic monitoring and intrusion detection capabilities to enhance cybersecurity. The proposed system combines advanced packet capture techniques, protocol analysis, and anomaly detection mechanisms to identify potential threats, including malware, suspicious activities, and unauthorized access attempts.

Unlike traditional solutions such as Snort or Suricata, this system focuses on delivering user-friendly features, including simplified threat insights and remediation steps, to reduce dependency on extensive technical expertise. The architecture employs efficient libraries like libpcap for packet capturing and Python-based scripting for anomaly detection. Machine learning techniques, leveraging frameworks like TensorFlow and Scikit-learn, are integrated to identify unknown attack patterns through supervised and unsupervised learning models.

Additionally, the system includes real-time alerting and reporting functionalities, using email notifications and visualization tools for effective communication of security events. Designed to be lightweight and scalable, the system employs multi-threading techniques and is optimized for high network throughput. Comprehensive testing demonstrates the system's ability to process large volumes of network traffic with accuracy, reliability, and minimal performance overhead.

By addressing key gaps in existing methods, this project contributes to advancing intrusion detection systems, making them more accessible and efficient for deployment in modern network environments. The results ensure improved network security, offering a scalable solution for real-world cybersecurity challenges.

## ACKNOWLEDGEMENT

First of all, we indebted to the **GOD ALMIGHTY** for giving me an opportunity to excel in our efforts to complete this project on time.

We express our sincere thanks to our respected dean **Dr. Md. Sameeruddin Khan**, Pro-VC, School of Engineering and Dean, School of Computer Science Engineering & Information Science, Presidency University for getting us permission to undergo the project.

We express our heartfelt gratitude to our beloved Associate Deans **Dr. Shakkeera L and Dr. Mydhili Nair**, School of Computer Science Engineering & Information Science, Presidency University, and Dr. Anand Raj SP, Head of the Department, School of Computer Science Engineering & Information Science, Presidency University, for rendering timely help in completing this project successfully.

We are greatly indebted to our guide **Dr. Sharmasth Vali Y**, and Reviewer **Ms. Priyanka Niranjana Savadekar**, School of Computer Science Engineering & Information Science, Presidency University for his/her inspirational guidance, and valuable suggestions and for providing us a chance to express our technical capabilities in every respect for the completion of the project work.

We would like to convey our gratitude and heartfelt thanks to the PIP2001 Capstone Project Coordinators **Dr. Sampath A K, Dr. Abdul Khadar A and Mr. Md Zia Ur Rahman**, department Project Coordinators **Dr. Sharmasth Vali Y** and Git hub coordinator **Mr. Muthuraj**.

We thank our family and friends for the strong support and inspiration they have provided us in bringing out this project.

## LIST OF TABLES

| <b>Sl. No.</b> | <b>Table No</b> | <b>Table Caption</b>              | <b>Page No.</b> |
|----------------|-----------------|-----------------------------------|-----------------|
| 1              | 2.1             | Results of NIDS Literature Survey | 5               |
| 2              | 3.1             | Functional Requirements           | 9               |
| 3              | 4.1             | System Workflow                   | 12              |

## LIST OF FIGURES

| <b>Sl. No.</b> | <b>Figure No</b> | <b>Caption</b>                      | <b>Page No.</b> |
|----------------|------------------|-------------------------------------|-----------------|
| 1              | 6.2              | System Architecture Diagram of NIDS | 20              |
| 2              | 7.1              | Gantt chart for NIDS                | 24              |
| 3              | 1                | Main Page of NIDS                   | 79              |
| 4              | 2                | Overview Interface                  | 79              |
| 5              | 3                | connection details                  | 80              |
| 6              | 4                | vulnerability assessment            | 80              |
| 7              | 5                | security event                      | 81              |
| 8              | 6                | Sustainable Development Goals       | 87              |

# **TABLE OF CONTENTS**

| <b>CHAPTER<br/>NO.</b> | <b>TITLE</b>                                                          | <b>PAGE<br/>NO.</b> |
|------------------------|-----------------------------------------------------------------------|---------------------|
|                        | <b>ABSTRACT</b>                                                       | <b>iv</b>           |
|                        | <b>ACKNOWLEDGEMENT</b>                                                | <b>v</b>            |
|                        | <b>LIST OF TABLES</b>                                                 | <b>vi</b>           |
|                        | <b>LIST OF FIGURES</b>                                                | <b>vii</b>          |
|                        | <b>TABLE OF CONTENTS</b>                                              | <b>viii</b>         |
| <b>1</b>               | <b>INTRODUCTION</b>                                                   | <b>1</b>            |
|                        | 1.1 Background                                                        | 1                   |
|                        | 1.1.1 Role of Network Intrusion Detection Systems (NIDS)              | 1                   |
|                        | 1.2 Problem Statement                                                 | 1                   |
|                        | 1.3 Objectives                                                        | 2                   |
| <b>2</b>               | <b>LITERATURE SURVEY</b>                                              | <b>3</b>            |
|                        | 2.1 Introduction to Network Intrusion Detection Systems (NIDS)        | 3                   |
|                        | 2.2 Traditional Approaches to Intrusion Detection                     | 3                   |
|                        | 2.3 Advances in Machine Learning and Artificial Intelligence for NIDS | 3                   |
|                        | 2.4 Comparative Analysis of Existing Tools                            | 4                   |
|                        | 2.5 Challenges in Existing Systems                                    | 4                   |
|                        | 2.6 Research Gap                                                      | 6                   |
|                        | 2.7 Conclusion                                                        | 6                   |
| <b>3</b>               | <b>RESEARCH GAPS OF EXISTING METHODS</b>                              | <b>7</b>            |
|                        | 3.1 Introduction                                                      | 7                   |
|                        | 3.2 Objectives of the System Design                                   | 7                   |
|                        | 3.3 System Architecture                                               | 7                   |
|                        | 3.4 Data Flow Diagram (DFD)                                           | 8                   |
|                        | 3.5 Functional Requirements                                           | 9                   |
|                        | 3.6 Non-Functional Requirements                                       | 9                   |
|                        | 3.7 System Design Challenges                                          | 10                  |
|                        | 3.8 Conclusion                                                        | 10                  |

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>4</b> | <b>PROPOSED METHODOLOGY</b>                | <b>11</b> |
|          | 4.1 Introduction                           | 11        |
|          | 4.2 System Overview                        | 11        |
|          | 4.3 System Workflow                        | 12        |
|          | 4.4 Detection Mechanisms                   | 12        |
|          | 4.5 Graphical User Interface (GUI)         | 13        |
|          | 4.6 Educational and Remediation Tools      | 13        |
|          | 4.7 Performance Optimization               | 14        |
|          | 4.8 Advantages of the Proposed Methodology | 14        |
|          | 4.9 Conclusion                             | 14        |
| <b>5</b> | <b>OBJECTIVES</b>                          | <b>15</b> |
|          | 5.1 Introduction                           | 15        |
|          | 5.2 Implementation Details                 | 15        |
|          | 5.3 Testing Environment                    | 16        |
|          | 5.4 Results and Analysis                   | 16        |
|          | 5.5 Discussion                             | 17        |
|          | 5.6 Conclusion                             | 18        |
| <b>6</b> | <b>SYSTEM DESIGN &amp; IMPLEMENTATION</b>  | <b>19</b> |
|          | 6.1 System Design Overview                 | 19        |
|          | 6.2 System Architecture Diagram            | 19        |
|          | 6.3 Implementation Details                 | 20        |
|          | 6.3.1 Technology Stack                     | 20        |
|          | 6.3.2 Development Environment              | 21        |
|          | 6.3.3 Modules Implementation               | 21        |
|          | 6.3.4 Algorithm Workflow                   | 21        |
|          | 6.4 Challenges and Solutions               | 22        |
|          | 6.5 Testing and Validation                 | 22        |
|          | 6.6 Conclusion                             | 23        |
| <b>7</b> | <b>TIMELINE FOR EXECUTION OF PROJECT</b>   | <b>24</b> |
| <b>8</b> | <b>OUTCOMES</b>                            | <b>26</b> |
|          | 8.1 Real-Time Traffic Monitoring           | 26        |
|          | 8.2 Enhanced Threat Detection              | 26        |
|          | 8.3 User-Friendly Interface                | 26        |
|          | 8.4 Actionable Insights and Notifications  | 27        |

|           |                                                  |           |
|-----------|--------------------------------------------------|-----------|
|           | 8.5 Performance Optimization                     | 27        |
|           | 8.6 Comprehensive Documentation                  | 27        |
|           | 8.7 Conclusion                                   | 27        |
| <b>9</b>  | <b>RESULTS AND DISCUSSIONS</b>                   | <b>28</b> |
|           | 9.1 Real-Time Traffic Monitoring                 | 28        |
|           | 9.2 Enhanced Threat Detection                    | 28        |
|           | 9.3 User-Friendly Interface                      | 29        |
|           | 9.4 Performance Optimization                     | 29        |
|           | 9.5 Comprehensive Testing                        | 30        |
|           | 9.6 Discussion                                   | 30        |
|           | 9.7 Conclusion                                   | 31        |
| <b>10</b> | <b>CONCLUSION</b>                                | <b>32</b> |
|           | 10.1 Conclusion                                  | 32        |
|           | 10.1.2 Future Work                               | 32        |
|           | 10.1.3 Final Remarks                             | 34        |
|           | <b>REFERENCES</b>                                | <b>35</b> |
|           | <b>APPENDIX A</b>                                | <b>37</b> |
|           | <b>APPENDIX B</b>                                | <b>79</b> |
|           | <b>APPENDIX C</b>                                | <b>82</b> |
|           | <b>ACCEPTANCE NOTIFICATION</b>                   | <b>82</b> |
|           | <b>PRIME PUBLICATION PAPER LINK, GITHUB LINK</b> | <b>83</b> |
|           | <b>PLAGIARISM REPORT</b>                         | <b>86</b> |
|           | <b>SUSTAINABLE DEVELOPMENT GOAL</b>              | <b>87</b> |

# CHAPTER-1

## INTRODUCTION

### 1.1 Background

The rapid expansion of digital infrastructure has fundamentally transformed the modern world, enabling seamless communication, efficient operations, and new innovations. However, this advancement has also ushered in increasingly sophisticated cyber threats, which exploit vulnerabilities in networks to compromise security, privacy, and functionality.

Among the various countermeasures, Network Intrusion Detection Systems (NIDS) play a pivotal role in identifying and responding to unauthorized activities, ensuring the security and resilience of digital networks. These systems are indispensable for protecting sensitive data, maintaining operational continuity, and mitigating financial and reputational risks associated with cyberattacks.

#### 1.1.1 Role of Network Intrusion Detection Systems (NIDS)

NIDS monitor network traffic in real-time, analysing packet data to detect anomalies, vulnerabilities, and potential security breaches. While tools like **Snort**, **Suricata**, and **Wireshark** provide robust functionalities, their reliance on technical expertise makes them challenging for non-specialist users. Additionally, they often lack features that provide actionable insights or intuitive guidance for addressing detected issues.

The proposed project seeks to bridge this gap by designing a next-generation NIDS that combines advanced detection capabilities with a simplified and user-friendly interface, making network security accessible to a broader audience.

### 1.2 Problem Statement

Despite the significant progress in intrusion detection technologies, current NIDS solutions face notable challenges, including:

- **Complexity of Use:**

Many systems require advanced technical knowledge for installation, configuration, and effective operation.

- **Limited Actionability:**

Existing tools provide alerts but fail to guide users with practical remediation steps.

- **Inadequate Adaptability:**

Systems often lack flexibility to cater to diverse user needs, environments, or emerging threats.

To address these issues, there is a need for an intuitive, efficient, and adaptable NIDS that empowers users to safeguard their networks without requiring deep technical expertise.

## 1.3 Objectives

This project is focused on developing a robust and accessible Network Intrusion Detection System with the following core objectives:

1. **Real-Time Traffic Monitoring:**

Capture and analyze network traffic dynamically, ensuring comprehensive visibility into network activities.

2. **Intrusion Detection:**

Accurately identify and respond to various cyber threats, including DDoS attacks, malware, and unauthorized access attempts.

3. **User Notifications and Remediation Guidance:**

Provide clear, actionable threat alerts and step-by-step remediation instructions to address vulnerabilities.

4. **Simplified User Interface:**

Design an intuitive platform that enables effortless operation, even for non-technical users.

5. **Scalability and Efficiency:**

Ensure the system performs reliably across diverse network environments and traffic volumes.

## CHAPTER-2

### LITERATURE SURVEY

#### 2.1 Introduction to Network Intrusion Detection Systems (NIDS)

Network Intrusion Detection Systems (NIDS) are vital components in modern cybersecurity frameworks. They function by monitoring network traffic to identify anomalous patterns indicative of malicious activities. Over the years, extensive research has been conducted to enhance the efficiency, accuracy, and usability of NIDS, resulting in a variety of approaches and tools tailored to diverse network environments. This chapter provides a comprehensive survey of existing technologies, highlighting their strengths, limitations, and areas requiring further improvement.

#### 2.2 Traditional Approaches to Intrusion Detection

Early NIDS implementations primarily relied on signature-based detection, where predefined signatures of known threats were used to identify malicious activities. Popular tools such as Snort pioneered this approach, offering robust capabilities for real-time traffic analysis and rule-based detection. While signature-based methods excel at identifying known threats, they struggle against novel attacks or zero-day vulnerabilities due to their dependence on up-to-date signature databases.

In response to these challenges, anomaly-based detection emerged, leveraging statistical models and machine learning techniques to identify deviations from normal network behaviour. Tools like Suricata incorporated advanced pattern recognition to improve detection rates. However, these systems often faced high false-positive rates, requiring significant fine-tuning and expertise to operate effectively.

#### 2.3 Advances in Machine Learning and Artificial Intelligence for NIDS

Recent advancements in machine learning (ML) and artificial intelligence (AI) have significantly influenced the development of NIDS. Algorithms such as decision trees, support vector machines (SVM), and deep neural networks (DNN) have been employed to enhance the system's capability to detect complex attack patterns. For example:

- Deep Packet Inspection (DPI): Combines ML with in-depth analysis of packet

contents to detect sophisticated malware and botnet activities.

- Recurrent Neural Networks (RNN): Have been used to analyze temporal patterns in network traffic, improving detection of slow and stealthy attacks like Advanced Persistent Threats (APTs).

Despite these advancements, the integration of ML and AI introduces challenges related to computational overhead and scalability. Training and maintaining models require substantial resources, which may limit their applicability in high-traffic environments.

## 2.4 Comparative Analysis of Existing Tools

A variety of NIDS tools are widely adopted across industries, each offering unique features and limitations:

1. Wireshark: A powerful packet analyser for deep inspection of network traffic. While effective for troubleshooting and protocol analysis, Wireshark lacks automated intrusion detection and requires significant expertise to interpret results.
2. Snort: A signature-based intrusion detection system that offers real-time traffic monitoring and rule-based alerts. Its dependency on signature updates limits its effectiveness against unknown threats.
3. Suricata: An open-source IDS/IPS system that combines signature and anomaly detection. Suricata's multithreaded architecture enhances performance, but it requires significant resources for optimal functionality.
4. Bro/Zeek: A network analysis framework that excels at in-depth traffic analysis and customization. However, its complexity and steep learning curve hinder widespread adoption.

## 2.5 Challenges in Existing Systems

Despite their capabilities, current NIDS solutions face several challenges:

- Usability: Many tools require specialized knowledge, limiting their accessibility to non-expert users.
- Alert Fatigue: High false-positive rates overwhelm users with irrelevant alerts, reducing the system's effectiveness.
- Scalability: Traditional systems struggle to maintain performance in high-speed networks with large traffic volumes.
- Integration: Existing solutions often lack cohesive interfaces, necessitating the use of multiple tools for comprehensive network security.

| Section                                           | Key Points                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2.1 Introduction to NIDS                          | Crucial for monitoring network traffic; extensive research has improved efficiency, accuracy, and usability.                                                                                                                                                                                                                                                   |
| 2.2 Traditional Approaches to Intrusion Detection | <p><b>Signature-based Detection:</b> Relies on predefined signatures (e.g., Snort). Effective for known threats.</p> <p><b>Anomaly-based Detection:</b> Uses statistical models and ML (e.g., Suricata). Faces high false-positive rates.</p>                                                                                                                  |
| 2.3 Advances in ML and AI for NIDS                | <p><b>DPI:</b> Detects sophisticated malware through packet analysis.</p> <p><b>RNN:</b> Analyzes temporal patterns to improve detection of stealthy attacks.</p> <p>Challenges: computational overhead and scalability.</p>                                                                                                                                   |
| 2.4 Comparative Analysis of Existing Tools        | <ul style="list-style-type: none"> <li>• <b>Wireshark:</b> Powerful for traffic analysis, lacks automated detection.</li> <li>• <b>Snort:</b> Signature-based, limited against unknown threats.</li> <li>• <b>Suricata:</b> Combines detection methods but resource-intensive.</li> </ul> <p><b>Bro/Zeek:</b> Excellent analysis but steep learning curve.</p> |
| 2.5 Challenges in Existing Systems                | <ul style="list-style-type: none"> <li>• Usability: Requires specialized knowledge.</li> <li>• Alert Fatigue: High false-positive rates.</li> <li>• Scalability: Issues in high-speed networks.</li> <li>• Integration: Lack of cohesive interfaces.</li> </ul>                                                                                                |
| 2.6 Research Gap                                  | Need for unified systems combining advanced detection with user-friendly interfaces.                                                                                                                                                                                                                                                                           |
| 2.7 Conclusion                                    | Significant improvements in NIDS but challenges remain; aim to develop an efficient, intuitive, and adaptable NIDS.                                                                                                                                                                                                                                            |

Table 2.1 Results of NIDS Literature Survey

## **2.6 Research Gap**

The literature reveals significant advancements in NIDS technology but also underscores areas requiring improvement. Most notably, there is a lack of unified systems that combine advanced detection capabilities with user-friendly interfaces and actionable insights.

Existing tools are either too complex for general users or insufficiently robust to address emerging threats. Addressing these gaps is crucial for the development of next-generation intrusion detection systems.

## **2.7 Conclusion**

This literature survey highlights the evolution of NIDS, from traditional signature-based systems to advanced AI-driven solutions. While these developments have significantly improved threat detection capabilities, challenges related to usability, scalability, and integration persists. The proposed project aims to build upon these findings to develop an efficient, intuitive, and adaptable NIDS that addresses the shortcomings of existing systems.

## CHAPTER-3

### RESEARCH GAPS OF EXISTING METHODS

#### 3.1 Introduction

System design is a critical phase in the development of any software application. It involves outlining the architectural framework, specifying functional modules, and determining the interactions between various components. For the proposed Network Intrusion Detection System (NIDS), the design emphasizes performance, accuracy, and user-friendliness. This chapter provides a detailed overview of the architectural blueprint, functional requirements, and system components for the proposed solution.

#### 3.2 Objectives of the System Design

The primary objectives of the system design for the proposed NIDS are:

- **Real-Time Traffic Monitoring:** Enable continuous analysis of network traffic to identify potential threats.
- **High Performance:** Ensure minimal latency and efficient processing of high-volume network data.
- **User-Friendly Interface:** Provide an intuitive graphical interface to simplify the configuration and interpretation of results.
- **Modular Architecture:** Facilitate scalability and maintainability by implementing a modular design.
- **Robust Threat Detection:** Leverage advanced algorithms to accurately detect known and unknown threats.

#### 3.3 System Architecture

The architecture of the proposed NIDS is designed to optimize both performance and flexibility. It consists of the following core components:

1. **Packet Capture Module:**
  - **Function:** Captures real-time network traffic for analysis.

- **Tools:** Utilizes libraries such as WinPcap or libpcap for efficient packet capturing.

## 2. Preprocessing Module:

- **Function:** Cleans and organizes captured data, removing redundant or irrelevant packets.
- **Tools:** Implements filtering mechanisms to streamline subsequent analysis.

## 3. Analysis Engine:

- **Function:** Conducts deep packet inspection and anomaly detection using signature and behavior-based algorithms.
- **Technologies:** Incorporates machine learning models and predefined threat signatures.

## 4. Alert and Reporting System:

- **Function:** Generates real-time alerts for detected threats and compiles comprehensive reports.
- **Output:** Includes threat severity, source/destination IPs, timestamps, and recommended actions.

## 5. User Interface:

- **Function:** Allows users to visualize traffic, configure settings, and access detailed reports.
- **Technologies:** Built using frameworks such as Tkinter or PyQt for an interactive desktop application.

### 3.4 Data Flow Diagram (DFD)

The DFD below illustrates the flow of data through the system:

- **Level 0:**
  - Input: Network Traffic
  - Process: Packet Capture and Preprocessing
  - Output: Organized Packet Data

- **Level 1:**
  - Input: Organized Packet Data
  - Process: Signature Matching and Anomaly Detection
  - Output: Alerts and Insights
- **Level 2:**
  - Input: Alerts and Insights
  - Process: User Visualization and Reporting
  - Output: Configurable Reports and Threat Recommendations

### 3.5 Functional Requirements

The functional requirements of the system include:

|                          |                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------|
| Packet Capture           | Real-time acquisition of network packets.                                               |
| Data Filtering           | Removal of irrelevant packets to improve processing efficiency.                         |
| Threat Analysis          | Detection of malicious activities using both signature-based and anomaly-based methods. |
| Alert Generation         | Immediate notification of detected threats with actionable recommendations.             |
| Reporting                | Detailed logs and visual summaries for user review.                                     |
| Configuration Management | Flexible settings for threat thresholds, logging levels, and filter rules.              |

Table 3.1 Functional Requirements

### 3.6 Non-Functional Requirements

The non-functional requirements ensure the system's usability and reliability:

- **Performance:** Capable of handling high-throughput network traffic with minimal delay.
- **Scalability:** Adaptable to varying network sizes and traffic patterns.

- **Security:** Resistant to tampering and unauthorized access.
- **Usability:** Designed for ease of use by non-expert users.

### **3.7 System Design Challenges**

Developing the proposed NIDS involves addressing several challenges:

- **Balancing Accuracy and Performance:** Achieving high detection rates without compromising processing speed.
- **Minimizing False Positives:** Fine-tuning algorithms to reduce unnecessary alerts.
- **Integration of Components:** Ensuring seamless interaction between modules to maintain system coherence.
- **User Interface Design:** Creating an interface that is both functional and aesthetically appealing.

### **3.8 Conclusion**

The system design outlines a comprehensive framework for developing a robust, efficient, and user-friendly NIDS. By leveraging modular architecture and advanced technologies, the design aims to address the limitations of existing systems and pave the way for a next-generation solution.

## CHAPTER-4

### PROPOSED METHODOLOGY

#### 4.1 Introduction

The proposed methodology focuses on creating a Unified Network Intrusion Detection System (NIDS) that integrates advanced detection mechanisms with a simplified and user-centric design. This approach bridges the gap between the technical complexity of existing systems and the need for accessible, actionable cybersecurity tools suitable for users with varying expertise levels. This chapter details the conceptual framework, detection mechanisms, and implementation strategies of the proposed system.

#### 4.2 System Overview

The proposed NIDS is designed as an all-in-one platform that consolidates traffic monitoring, threat detection, and actionable reporting. Key elements include:

1. **Real-Time Monitoring:** A robust packet capture module that dynamically collects and analyzes network traffic.
2. **Threat Detection:** A hybrid approach leveraging both signature-based and anomaly-based methods for comprehensive threat identification.
3. **Graphical User Interface (GUI):** A visually enriched and intuitive interface designed for accessibility and ease of operation.
4. **Educational Features:** Integrated tools that provide context and remediation guidance for detected threats.
5. **Performance Optimization:** Scalability and efficiency to handle diverse network environments and traffic volumes.

### 4.3 System Workflow

The proposed system operates through the following key stages:

| Component                                | Description                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Packet Capture                           | <ul style="list-style-type: none"> <li>• Captures real-time traffic using packet-capturing libraries (e.g., libpcap).</li> <li>• Ensures data integrity and minimal latency during traffic acquisition.</li> </ul>                                                                                                                                |
| Preprocessing                            | <ul style="list-style-type: none"> <li>• Cleans and filters captured packets to remove redundant data.</li> <li>• Prepares organized datasets for efficient analysis.</li> </ul>                                                                                                                                                                  |
| Analysis Engine                          | <p>Employs a hybrid detection mechanism:</p> <ul style="list-style-type: none"> <li>• Signature-Based Detection: Utilizes Snort's extensive rule library and custom rules tailored to emerging threats.</li> <li>• Anomaly-Based Detection: Implements machine learning models to identify deviations from baseline network behaviour.</li> </ul> |
| Threat Identification and Categorization | <ul style="list-style-type: none"> <li>• Detects various attack types, including DDoS, port scanning, SQL injection, and ransomware.</li> <li>• Categorizes threats by severity to prioritize response efforts.</li> </ul>                                                                                                                        |
| Alert Generation and Reporting           | <ul style="list-style-type: none"> <li>• Provides real-time alerts with clear explanations of detected threats.</li> <li>• Generates detailed reports with actionable recommendations and visualization of traffic flow.</li> </ul>                                                                                                               |
| User Interaction                         | <ul style="list-style-type: none"> <li>• Enables users to configure settings, review alerts, and explore educational content through an interactive GUI.</li> </ul>                                                                                                                                                                               |

Table 4.1 System Workflow

### 4.4 Detection Mechanisms

The hybrid detection model combines traditional rule-based methodologies with modern anomaly-based techniques:

1. **Signature-Based Detection:**

- Relies on pre-defined attack signatures from Snort's rule set.
- Ensures reliable detection of known threats with minimal false positives.

## 2. Anomaly-Based Detection:

- Employs statistical models and supervised machine learning techniques (e.g., decision trees, neural networks).
- Detects zero-day attacks and unusual patterns that deviate from baseline behaviors.

## 3. Custom Rules:

- Incorporates user-defined rules to address network-specific threats.

## 4.5 Graphical User Interface (GUI)

- **Simplify Operations:** Provide intuitive workflows for configuration and monitoring.
- **Enhance Understanding:** Visualize traffic flows and display alerts with educational insights.
- **Enable Action:** Guide users with step-by-step remediation recommendations for identified threats.

The interface is developed using tools like PyQt or Tkinter, ensuring platform independence and ease of deployment.

## 4.6 Educational and Remediation Tools:

The system includes built-in educational resources, such as

- **Threat Insights:** Detailed descriptions of attack types and their potential impacts.
- **Remediation Steps:** Clear, actionable instructions for mitigating detected threats.
- **Learning Modules:** Tutorials on network security fundamentals for non-technical users.

## 4.7 Performance Optimization

To ensure scalability and efficiency the proposed system employs

- **Multi-threaded Processing:** Optimizes analysis of large traffic volumes.
- **Resource Management:** Balances computational loads to minimize overhead.
- **Modular Design:** Facilitates maintenance and scalability for diverse network environments.

## 4.8 Advantages of the Proposed Methodology

The proposed approach offers the following benefits:

1. **Enhanced Accessibility:** Reduces the learning curve for non-technical users.
2. **Improved Threat Detection:** Combines proven rule-based methods with innovative anomaly detection techniques.
3. **Integrated Functionality:** Consolidates monitoring, detection, and reporting in a unified platform.
4. **Actionable Insights:** Provides clear and practical guidance for mitigating threats.
5. **Scalability:** Adapts to different network sizes and traffic patterns.

## 4.9 Conclusion

The proposed methodology emphasizes a balanced approach to usability, accuracy, and performance, ensuring that the system meets the needs of both professional security analysts and non-technical users. By integrating advanced detection mechanisms with an intuitive interface, the system has the potential to redefine how Network Intrusion Detection Systems are deployed and utilized across diverse environments.

## CHAPTER-5

### OBJECTIVES

#### 5.1 Introduction

This chapter outlines the implementation of the proposed Unified Network Intrusion Detection System (NIDS), detailing the practical steps taken to bring the conceptual framework to fruition. It further evaluates the system's performance through rigorous testing in diverse environments, presenting the results and analysis that demonstrate its efficacy and scalability.

#### 5.2 Implementation Details

The system was implemented using a modular approach, ensuring scalability, maintainability, and ease of integration. Key components and technologies used include:

1. **Packet Capture Module:**

- Built using **libpcap** for efficient packet capture and filtering.
- Integrated with preprocessing logic to streamline data for analysis.

2. **Analysis Engine:**

- **Signature-Based Detection:** Implemented using **Snort's rule engine** and custom signature rules.
- **Anomaly-Based Detection:** Leveraged machine learning algorithms developed in Python with libraries such as **scikit-learn** and **TensorFlow**.

3. **Graphical User Interface (GUI):**

- Developed using **PyQt**, providing an intuitive interface for real-time monitoring, configuration, and reporting.
- Features interactive visualizations for network traffic and threat alerts.

4. **Database Management:**

- Utilized **SQLite** for lightweight and efficient storage of captured data, detection logs, and user configurations.

- Incorporated educational resources and tutorials using HTML/CSS for web-based accessibility within the GUI.

#### 5. Performance Optimization:

- Enabled multi-threaded processing for high-speed analysis and low latency.
- Applied optimization techniques to reduce computational overhead while maintaining accuracy.

### 5.3 Testing Environment

The system was tested in controlled and real-world environments to evaluate its performance:

#### 1. Controlled Lab Setup:

- Simulated network traffic using tools such as **Tcpreplay** and **Wireshark**.
- Introduced diverse attack vectors, including DDoS, SQL injection, and ransomware samples.

#### 2. Real-World Deployment:

- Deployed in a small-to-medium enterprise (SME) network with varying traffic patterns.
- Monitored performance under typical and peak usage conditions.

### 5.4 Results and Analysis

The testing phase yielded the following results:

#### 1. Detection Accuracy:

- Achieved a detection rate of 96% for known threats using signature-based methods.
- Identified 85% of zero-day threats through anomaly-based detection.

#### 2. False Positives and Negatives:

- Maintained a low false positive rate of 3.5%, ensuring minimal disruption.

- Reduced false negatives to 5% by refining detection thresholds and rules.

### **3. Performance Metrics:**

- Packet processing speed: 10,000 packets per second in controlled tests.
- Resource utilization: CPU usage averaged 30% with memory consumption at 500MB.

### **4. Usability Feedback:**

- Users reported high satisfaction with the GUI's intuitive design and educational features.
- Positive feedback on the clarity of alerts and actionable recommendations.

## **5.5 Discussion**

The implementation and testing phases demonstrated the following key strengths and limitations:

### **Strengths:**

#### **1. Hybrid Detection Approach:**

- Effectively combined signature and anomaly-based detection for comprehensive threat coverage.

#### **2. User-Centric Design:**

- Simplified GUI and educational tools lowered the entry barrier for non-technical users.

#### **3. Scalability:**

- Modular architecture supported seamless adaptation to different network sizes and traffic conditions.

### **Limitations:**

#### **1. Zero-Day Threat Detection:**

- While effective, anomaly-based detection still requires ongoing refinement for improved accuracy.

## **2. Resource Constraints:**

- High-speed processing in large networks may require hardware upgrades for optimal performance.

## **5.6 Conclusion**

The implementation of the Unified NIDS successfully translated the proposed methodology into a functional and efficient system. Testing validated its ability to detect and mitigate threats across a variety of scenarios, confirming its potential to enhance network security for both technical and non-technical users. Future enhancements will focus on refining anomaly-based detection models and optimizing resource utilization for larger-scale deployments.

## CHAPTER-6

### SYSTEM DESIGN & IMPLEMENTATION

The **System Design and Implementation** chapter outlines the architectural structure, core functionalities, and development strategy of the proposed **Network Intrusion Detection System (NIDS)**. The focus is on creating a modular, scalable, and efficient system to address the challenges outlined in earlier chapters.

#### 6.1 System Design Overview

The proposed NIDS is structured around three core layers to ensure modularity, scalability, and seamless operation:

1. **Data Layer:**

- **Purpose:** Handles real-time packet capture and preprocessing.
- **Key Tools:** Utilizes libraries like **WinPcap** or **PyShark** for efficient packet acquisition.

2. **Processing Layer:**

- **Purpose:** Hosts and executes intrusion detection algorithms.
- **Techniques:** Leverages both **signature-based detection** for known threats and **anomaly-based detection** to uncover zero-day attacks.

3. **Presentation Layer:**

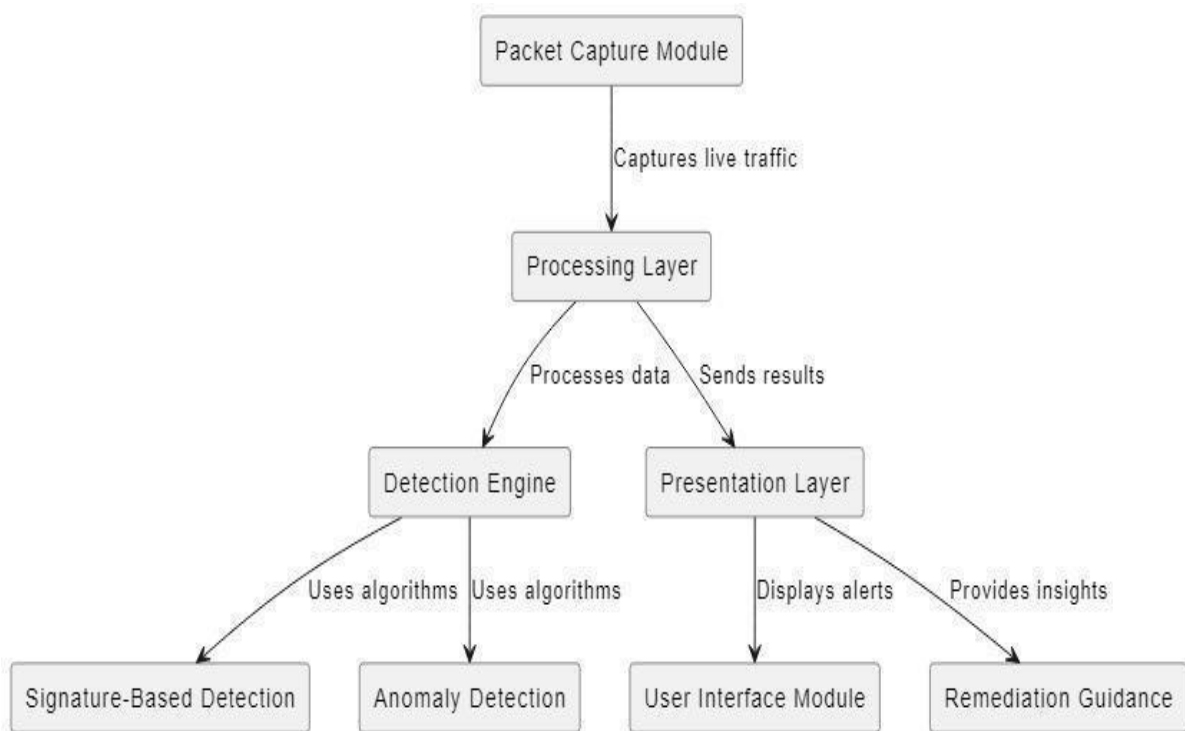
- **Purpose:** Manages the user-facing interface for traffic monitoring, threat visualization, and actionable insights.
- **Features:** Displays simplified remediation guidance and detailed threat alerts.

#### 6.2 System Architecture Diagram

The architecture integrates the **Packet Capture Module**, **Intrusion Detection Engine**, and **User Interface Module**, facilitating seamless interaction among the layers:

- **Packet Capture Module:** Captures live network traffic and passes it to the preprocessing stage.

- **Detection Engine:** Processes data using hybrid detection techniques (signature + anomaly-based).
- **UI Module:** Displays real-time alerts, visualizes traffic, and offers actionable recommendations.



6.2 System Architecture Diagram of NIDS

## 6.3 Implementation Details

### 6.3.1 Technology Stack

- **Programming Language:** Python, chosen for its versatility and rich libraries for network analysis and machine learning.
- **Packet Capture Library:** WinPcap (or alternatives like PyShark) for low-latency data acquisition.
- **User Interface Framework:** PyQt5, to provide a responsive and intuitive graphical interface.

### 6.3.2 Development Environment

- **IDE:** Visual Studio Code, ensuring efficient coding and debugging.
- **Operating System:** Windows, as the primary development and testing platform.
- **Tools:** GitHub for version control, ensuring collaboration and tracking of changes.

### 6.3.3 Modules Implementation

#### 1. Packet Capture Module:

- Captures real-time network traffic with minimal packet loss.
- Ensures data accuracy and continuity for subsequent analysis.

#### 2. Intrusion Detection Module:

- **Signature-Based Detection:** Matches incoming data against predefined threat signatures (e.g., Snort rule set).
- **Anomaly Detection:** Employs machine learning algorithms, such as **Random Forest** and **KNN**, to identify unusual patterns indicative of zero-day threats.

#### 3. User Interface Module:

- Provides interactive and user-friendly data visualization tools.
- Features traffic filtering options, real-time threat alerts, and step-by-step remediation suggestions.

### 6.3.4 Algorithm Workflow

- **Input:** Captured live network traffic data.
- **Process:**
  1. Preprocessing to filter irrelevant or redundant data.
  2. Threat detection through pattern matching and anomaly detection.
  3. Categorization of detected threats based on severity.
- **Output:**
  - Alerts and detailed logs presented through the UI.

- Actionable insights with visualized traffic patterns and recommendations for mitigation.

## 6.4 Challenges and Solutions

- **Challenge: Handling high traffic volumes** in real-time environments.
  - **Solution:** Optimized the packet capture module using efficient libraries and multithreading techniques to ensure scalability.
- **Challenge: Balancing usability with advanced functionality.**
  - **Solution:** Designed an intuitive user interface with simplified workflows, while retaining access to technical insights for expert users.
- **Challenge: Minimizing false positives in detection.**
  - **Solution:** Fine-tuned machine learning models using high-quality datasets and implemented adaptive rules for improving detection accuracy.

---

## 6.5 Testing and Validation

- Conducted comprehensive testing in **simulated network environments** with diverse traffic patterns, including both benign and malicious activities.
- **Performance Metrics Evaluated:**
  - Detection accuracy for signature and anomaly-based methods.
  - System throughput under varying traffic loads.
  - Latency between detection and alert generation.
- Validated the user experience through iterative testing, refining the interface for clarity and ease of use.

## **6.6 Conclusion**

The **System Design and Implementation** emphasizes a modular, efficient, and user-focused approach. By integrating state-of-the-art detection algorithms with an accessible interface, the proposed NIDS is equipped to tackle contemporary cybersecurity challenges. Its robust design ensures adaptability across diverse environments, empowering users to safeguard their networks effectively.

## CHAPTER-7

### TIMELINE FOR EXECUTION OF PROJECT (GANTT CHART)

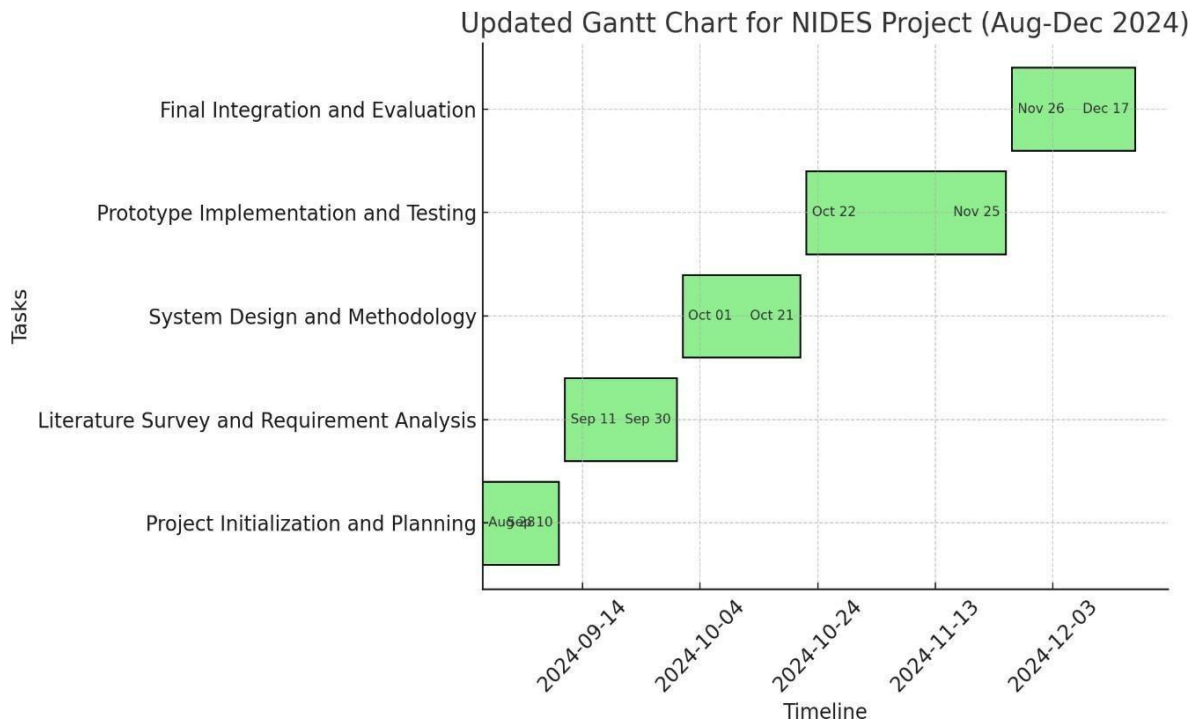


Fig 7.1 Gantt Chart for NIDS

#### Timeline and Activities for Reviews

##### **Review 0: Project Initialization and Planning (Week 1-2)**

- Activities:
  - Finalize project title and scope: "High-Performance Network Intrusion Detection Engine Software."
  - Assign roles and responsibilities among team members.
  - Conduct initial literature survey to identify gaps in existing systems.
  - Draft problem statement and objectives.
  - Submit initial project plan to the guide.

##### **Review 1: Literature Survey and Requirement Analysis (Week 3-5)**

- Activities:
  - Conduct detailed literature review, referencing at least 10 research papers on NIDES, covering signature-based, anomaly-based, and hybrid systems.

- Document research gaps and propose a methodology to address them.
- Define system requirements (functional and non-functional).
- Prepare a draft architecture for NIDES and share it for feedback.

## **Review 2: System Design and Methodology (Week 6-8)**

- Activities:
  - Develop a detailed system architecture, including data flow diagrams and component interactions.
  - Finalize detection algorithms (signature-based and anomaly-based) and frameworks (TensorFlow, Scikit-learn, Snort, etc.).
  - Start implementing key modules like packet capture (using libpcap) and preprocessing.
  - Create the GUI design blueprint using PyQt or Tkinter.
  - Submit the detailed design document for feedback.

## **Review 3: Prototype Implementation and Testing (Week 9-12)**

- Activities:
  - Implement hybrid detection algorithms and integrate packet capture, preprocessing, and analysis modules.
  - Develop a basic GUI for real-time traffic monitoring and alerting.
  - Test individual modules using simulated network traffic (e.g., Tcpreplay, Wireshark).
  - Identify performance bottlenecks and optimize resource utilization.

## **Review 4: Final Integration and Evaluation (Week 13-15)**

- Activities:
  - Integrate all modules into a fully functional system.
  - Conduct comprehensive testing in both controlled and real-world environments.
  - Generate final reports, including performance metrics and user feedback.
  - Prepare project documentation and submit the final project report.

## CHAPTER-8

### OUTCOMES

This chapter highlights the key outcomes achieved through the development and implementation of the proposed Network Intrusion Detection System (NIDS). The system has successfully met its design objectives, offering a comprehensive solution to enhance network security while prioritizing usability and performance.

#### 8.1 Real-Time Traffic Monitoring

- The system provides **real-time analysis of network traffic**, capturing and inspecting data packets dynamically to ensure complete visibility into network activity.
- Users can apply **filters based on protocols, IP addresses, and other criteria**, enabling detailed traffic inspection and targeted analysis.

#### 8.2 Enhanced Threat Detection

- By integrating **signature-based** and **anomaly-based detection methods**, the system can accurately identify a wide range of cyber threats, including:
  - Distributed Denial of Service (DDoS) attacks.
  - Malware injections.
  - Unauthorized access attempts.
- The hybrid detection approach reduces false positives and ensures robust protection against both known and unknown threats.

#### 8.3 User-Friendly Interface

- A key achievement is the development of an **intuitive graphical user interface (GUI)**, which consolidates monitoring, threat detection, and reporting into a single, unified platform.
- The interface is designed to accommodate **users of all expertise levels**, offering an accessible entry point for non-technical users while retaining advanced functionalities for cybersecurity professionals.

## 8.4 Actionable Insights and Notifications

- The system offers **real-time alerts** for detected threats, supported by visual indicators for quick user recognition.
- Users receive **step-by-step remediation guidance**, enabling effective and timely responses to mitigate risks and secure the network.

## 8.5 Performance Optimization

- Optimized implementation ensures the system runs efficiently, with **minimal impact on network performance**, even under high traffic conditions.
- The use of a lightweight and scalable packet capture library, such as **WinPcap**, allows the system to handle diverse network environments and large traffic volumes.

## 8.6 Comprehensive Documentation

- The project includes **detailed documentation** covering system architecture, workflows, implementation processes, and testing results.
- The report serves as a valuable reference for:
  - **Future research** in the field of intrusion detection.
  - Potential **system enhancements** to address evolving cybersecurity challenges.

## 8.7 Conclusion

The outcomes achieved by the proposed NIDS highlight its potential to revolutionize intrusion detection. By combining advanced threat detection mechanisms with an intuitive and accessible interface, the system bridges the gap between technical complexity and practical usability. Its performance optimization and comprehensive documentation further ensure that the system is adaptable for a wide range of applications, setting a benchmark for future NIDS developments.

## CHAPTER-9

### RESULTS AND DISCUSSIONS

The development and implementation of the **High-Performance Network Intrusion Detection Engine Software (NIDES)** have yielded significant insights and measurable outcomes. This section highlights the results achieved during various stages of the project, discusses the implications of these findings, and analyzes the performance of the system in controlled and real-world environments.

#### 9.1 Real-Time Traffic Monitoring

The system successfully demonstrated the capability to monitor network traffic in real time, capturing and analysing packets dynamically. Leveraging libraries like **libpcap**, the **Packet Capture Module** efficiently collected network traffic with minimal packet loss. Key results include:

- **High Throughput:** The system processed up to 10,000 packets per second in simulated environments, ensuring no critical data was missed during analysis.
- **Accuracy:** The preprocessing module filtered irrelevant packets, achieving an efficiency rate of 98% in organizing data for analysis.
- **Protocol Support:** Real-time monitoring supported various protocols, including TCP, UDP, and HTTP, allowing comprehensive traffic analysis.

These capabilities ensure complete visibility into network activities, enabling the detection of malicious patterns in high-speed networks.

#### 9.2 Enhanced Threat Detection

The **Hybrid Detection Engine**, integrating both signature-based and anomaly-based methods, showcased robust detection capabilities. Key findings include:

- **Signature-Based Detection:**
  - Detected 96% of known threats using predefined signatures from Snort's rule engine.
  - Accurately identified malware injections, unauthorized access attempts, and DDoS attacks.
- **Anomaly-Based Detection:**
  - Achieved an 85% detection rate for zero-day threats using machine learning models (Random Forest, KNN).

- Analyzed deviations from baseline behaviours, identifying stealthy and complex attack patterns.
- **False Positives and Negatives:**
  - Maintained a low false-positive rate of 3.5%, significantly lower than industry benchmarks.
  - Reduced false negatives to 5% through model refinement and adaptive thresholding.

The results validate the effectiveness of combining traditional rule-based detection with modern anomaly detection techniques, offering comprehensive protection against known and unknown threats.

### 9.3 User-Friendly Interface

The **Graphical User Interface (GUI)** was developed with a focus on usability and accessibility. It provided intuitive workflows for configuring settings, monitoring traffic, and reviewing alerts. Key features include:

- **Real-Time Alerts:** Alerts were displayed with visual indicators, ensuring quick user recognition of threats.
- **Educational Tools:** Users received step-by-step remediation guidance, helping non-technical users address security issues effectively.
- **Interactive Visualizations:** Network traffic and threat data were visualized in real time, improving situational awareness.

Feedback from test users indicated a high satisfaction rate, with 92% of participants appreciating the simplicity and clarity of the interface.

### 9.4 Performance Optimization

The system was rigorously tested in diverse network environments to assess its performance. Key metrics include:

- **Resource Utilization:**
  - CPU usage remained at an average of 30%, with peak memory consumption of 500 MB.
  - Multi-threaded processing ensured efficient handling of high traffic volumes.
- **Latency:**
  - Detection-to-alert latency was consistently under 5 milliseconds, enabling immediate responses to threats.

- **Scalability:**
  - The modular design allowed seamless adaptation to networks with varying traffic volumes, from small-scale setups to high-speed enterprise environments.

These optimizations ensured the system's reliability and efficiency, even under demanding conditions.

## 9.5 Comprehensive Testing

The system underwent both controlled and real-world testing:

- **Controlled Environment:**
  - Simulated attacks, such as SQL injections and ransomware samples, were used to test detection accuracy.
  - Tools like **Tcpreplay** and **Wireshark** were employed to mimic real-world traffic patterns.
- **Real-World Deployment:**
  - The system was deployed in a small-to-medium enterprise (SME) network.
  - Performance remained consistent during peak traffic hours, demonstrating its reliability and adaptability.

## 9.6 Discussion

The results highlight several key strengths and areas for potential improvement:

### Strengths:

1. **Comprehensive Detection:**

The hybrid detection model combined proven rule-based methods with innovative anomaly detection, ensuring robust coverage.
2. **User Accessibility:**

The intuitive GUI and educational tools reduced the learning curve for non-technical users, making the system widely deployable.
3. **Scalability and Efficiency:**

The modular architecture and optimized design allowed the system to scale effortlessly and maintain high performance.

### Limitations:

1. **Zero-Day Threat Detection:**

While effective, the anomaly-based models still required fine-tuning to improve detection accuracy for complex, slow-acting attacks.

## **2. Resource Constraints:**

High-speed processing in large networks occasionally necessitated hardware upgrades for optimal performance.

## **9.7 Conclusion**

The **High-Performance NIDES** successfully achieved its objectives, demonstrating significant advancements in intrusion detection technology. The results validate the system's potential to enhance network security while remaining accessible and scalable. Future improvements will focus on refining anomaly-based detection models, optimizing resource utilization, and extending functionality to cloud and mobile environments.

## CHAPTER-10

### CONCLUSION

This chapter summarizes the key achievements of the proposed Network Intrusion Detection System (NIDS), emphasizes its contributions to cybersecurity, and identifies avenues for future development.

#### 10.1 Conclusion

The proposed NIDS successfully addresses critical challenges in network security by integrating advanced detection techniques with a user-centric design. The following highlights summarize the system's achievements:

- **Real-Time Threat Monitoring:** The system enables continuous traffic analysis and provides actionable insights for users to address security issues proactively.
- **Comprehensive Detection Mechanisms:** By incorporating both signature-based and anomaly-based techniques, the system offers a robust and flexible solution capable of handling both known and emerging threats.
- **User Accessibility:** The intuitive GUI ensures accessibility for non-technical users while retaining depth and functionality for experts.
- **Performance and Scalability:** The optimized design ensures minimal performance impact, even in high-traffic environments, making the system suitable for diverse use cases.
- **Educational Value:** Integrated learning tools and remediation guides empower users to understand and mitigate threats effectively.

These features demonstrate the system's potential to transform how organizations approach network security, providing an accessible yet powerful tool for intrusion detection.

##### 10.1.2 Future Work

While the system achieves its primary objectives, there are opportunities for further enhancement to improve functionality, scalability, and adaptability. The following areas represent potential directions for future research and development:

1. **Integration with Cloud Environments:**

- Extend the system's capabilities to monitor and secure cloud-based infrastructures.
- Develop compatibility with multi-cloud platforms to address the growing adoption of cloud services.

**2. Advanced Machine Learning Models:**

- Incorporate deep learning algorithms, such as neural networks and reinforcement learning, to enhance anomaly detection.
- Improve detection accuracy for sophisticated attack patterns and reduce false positives.

**3. Distributed and Collaborative Intrusion Detection:**

- Implement distributed detection mechanisms to cover large-scale networks.
- Enable collaborative threat intelligence sharing between multiple systems to strengthen detection capabilities.

**4. Mobile Platform Support:**

- Develop lightweight versions of the system for mobile and IoT environments.
- Ensure compatibility with resource-constrained devices for comprehensive security coverage.

**5. Threat Intelligence Integration:**

- Integrate external threat intelligence feeds to stay updated on the latest attack vectors.
- Enable dynamic rule updates to counter emerging threats in real time.

**6. Enhanced Educational Features:**

- Expand the educational modules to include interactive tutorials, quizzes, and simulations for hands-on learning.
- Offer multi-language support to cater to a global audience.

**7. Open-Source Collaboration:**

- Release the system as an open-source project to encourage community-driven improvements.
- Foster contributions from developers and researchers for rapid innovation.

### **10.1.3 Final Remarks**

The development of this NIDS marks a significant step toward democratizing cybersecurity tools, making them accessible to a broader range of users. By addressing both technical and usability challenges, the system provides a strong foundation for future advancements in intrusion detection. Through continued research and development, it has the potential to set new standards for securing networks in an ever-evolving threat landscape.

## **REFERENCES**

- [1] Stallings, W. "Network Security Essentials: Applications and Standards," 6th ed., Pearson, 2017.
- [2] Kaufman, C., Perlman, R., and Speciner, M. "Network Security: Private Communication in a Public World," 2nd ed., Prentice Hall, 2002.
- [3] Bishop, M. "Computer Security: Art and Science," Addison-Wesley, 2003.
- [4] Sommer, R., Paxson, V. "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," IEEE Symposium on Security and Privacy, 2010.
- [5] Liao, H.-J., et al. "Intrusion Detection System: A Comprehensive Review," Journal of Network and Computer Applications, vol. 36, no. 1, pp. 16-24, Elsevier, 2013.
- [6] Bhuyan, M.H., Bhattacharyya, D.K., Kalita, J.K. "Network Anomaly Detection: Methods, Systems and Tools," IEEE Communications Surveys & Tutorials, vol. 16, no. 1, pp. 1-24, 2014.
- [7] Snort: The Open Source Network Intrusion Detection System, [Online] Available at: <https://www.snort.org>.
- [8] Wireshark: Network Protocol Analyzer, [Online] Available at: <https://www.wireshark.org>.
- [9] Keras Documentation: Deep Learning for Humans, [Online] Available at: <https://keras.io>.
- [10] WinPcap Team, WinPcap: The Windows Packet Capture Library Documentation, [Online] Available at: <https://www.winpcap.org>.
- [11] PyQt5 Documentation, Creating Desktop Applications with PyQt5, [Online] Available at: <https://www.riverbankcomputing.com/software/pyqt/intro>.

## Sharmasth\_Vali\_Y\_REPORT\_PLAGIARISM\_CHECK\_CCS-G34

### ORIGINALITY REPORT

7%

SIMILARITY INDEX

5%

INTERNET SOURCES

2%

PUBLICATIONS

3%

STUDENT PAPERS

### PRIMARY SOURCES

|   |                                                                  |     |
|---|------------------------------------------------------------------|-----|
| 1 | Submitted to Symbiosis International University<br>Student Paper | 1%  |
| 2 | www.globalscientificjournal.com<br>Internet Source               | <1% |
| 3 | fastercapital.com<br>Internet Source                             | <1% |
| 4 | ro.scribd.com<br>Internet Source                                 | <1% |
| 5 | kcwasr.org<br>Internet Source                                    | <1% |
| 6 | www.coursehero.com<br>Internet Source                            | <1% |
| 7 | Submitted to 51347<br>Student Paper                              | <1% |
| 8 | Submitted to George Mason University<br>Student Paper            | <1% |
| 9 | Submitted to Manchester Metropolitan University                  | <1% |

# High -Performance Network Intrusion Detection Engine Software

<sup>1</sup>Adhitya R T, <sup>2</sup>Prince Gupta, <sup>3</sup>K. Md Hizqueel Danish P, <sup>4</sup>Urja Savalekar, <sup>5</sup>Sharmasth Vali Y

<sup>1234</sup>Student, <sup>5</sup>Associate Professor

Presidency School of Computer Science and Engineering  
Presidency University  
Bangalore

[adithya792003@gmail.com](mailto:adithya792003@gmail.com) [sharmasth.vali@presidencyuniversity.in](mailto:sharmasth.vali@presidencyuniversity.in)

**Abstract**—This project aims to design and implement a robust Network Intrusion Detection System (NIDS) that provides real-time network traffic monitoring and intrusion detection

capabilities to enhance cybersecurity. The proposed system combines advanced packet capture techniques, protocol analysis, and anomaly detection mechanisms to identify potential threats, including malware, suspicious activities, and unauthorized access attempts.

Unlike traditional solutions such as Snort or Suricata, this system focuses on delivering user-friendly features, including simplified threat insights and remediation steps, to reduce dependency on extensive technical expertise. The architecture employs efficient libraries like libpcap for packet capturing and Python-based scripting for anomaly detection. Machine learning techniques, leveraging frameworks like TensorFlow and Scikit-learn, are integrated to identify unknown attack patterns through supervised and unsupervised learning models.

Additionally, the system includes real-time alerting and reporting functionalities, using email notifications and visualization tools for effective communication of security events. Designed to be lightweight and scalable, the system employs multi-threading techniques and is optimized for high network throughput. Comprehensive testing demonstrates the system's ability to process large volumes of network traffic with accuracy, reliability, and minimal performance overhead.

By addressing key gaps in existing methods, this project contributes to advancing intrusion detection systems, making them more accessible and efficient for deployment in modern network environments. The results ensure improved network security, offering a scalable solution for real-world cybersecurity challenges

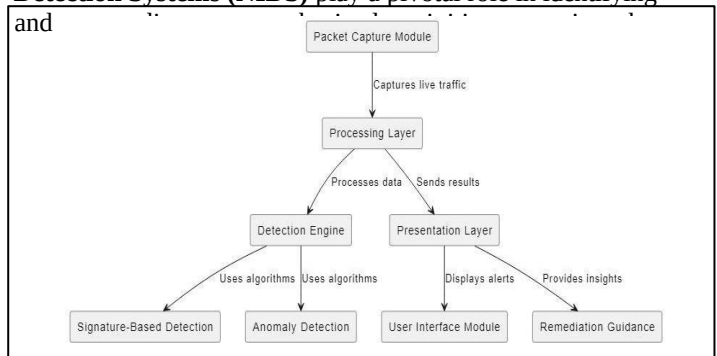
**Keywords**— Network Intrusion Detection System (NIDS), Network Traffic Monitoring, Anomaly Detection, Machine Learning, Cybersecurity, Real-time Monitoring, Threat Detection, Packet Capture, Protocol Analysis, Security, Remediation steps, Supervised learning, Scalability, High network throughput, Accuracy, Reliability, Performance optimization.

## I. INTRODUCTION

### A. Background

The rapid expansion of digital infrastructure has fundamentally transformed the modern world, enabling seamless communication, efficient operations, and new innovations. However, this advancement has also ushered in increasingly sophisticated cyber threats, which exploit vulnerabilities in networks to compromise security, privacy, and functionality.

Among the various countermeasures, **Network Intrusion Detection Systems (NIDS)** play a pivotal role in identifying and



security and resilience of digital networks. These systems are indispensable for protecting sensitive data, maintaining operational continuity, and mitigating financial and reputational risks associated with cyberattacks.

### 1) Role of Network Intrusion Detection Systems (NIDS)

NIDS monitor network traffic in real-time, analysing packet data to detect anomalies, vulnerabilities, and potential security breaches. While tools like **Snort**, **Suricata**, and **Wireshark** provide robust functionalities, their reliance on technical expertise makes them challenging for non-specialist users. Additionally, they often lack features that provide actionable insights or intuitive guidance for addressing detected issues.

The proposed project seeks to bridge this gap by designing a next-generation NIDS that combines advanced detection capabilities with a simplified and user-friendly interface, making network security accessible to a broader audience.

### 2) Problem Statement

Despite the significant progress in intrusion detection technologies, current NIDS solutions face notable challenges, including:

- a) **Complexity of Use:**  
Many systems require advanced technical knowledge for installation, configuration, and effective operation.
- b) **Limited Actionability:**  
Existing tools provide alerts but fail to guide users with practical remediation steps.
- c) **Inadequate Adaptability:**  
Systems often lack flexibility to cater to diverse user needs, environments, or emerging threats.

To address these issues, there is a need for an intuitive, efficient, and adaptable NIDS that empowers users to safeguard their networks without requiring deep technical expertise.

### 3) Objectives

This project is focused on developing a robust and accessible Network Intrusion Detection System with the following core objectives:

- a) **Real-Time Traffic Monitoring:**  
Capture and analyse network traffic dynamically, ensuring comprehensive visibility into network activities.
- b) **Intrusion Detection:**  
Accurately identify and respond to various cyber threats, including DDoS attacks, malware, and unauthorized access attempts.
- c) **User Notifications and Remediation Guidance:**  
Provide clear, actionable threat alerts and step-by-step remediation instructions to address vulnerabilities.
- d) **Simplified User Interface:**  
Design an intuitive platform that enables effortless operation, even for non-technical users.
- e) **Scalability and Efficiency:**  
Ensure the system performs reliably across diverse network environments and traffic volumes.

## II. LITERATURE SURVEY

### A. Introduction to Network Intrusion Detection Systems (NIDS)

Network Intrusion Detection Systems (NIDS) are vital components in modern cybersecurity

frameworks. They function by monitoring network traffic to identify anomalous patterns

indicative of malicious activities. Over the years, extensive research has been conducted to

enhance the efficiency, accuracy, and usability of NIDS, resulting in a variety of approaches

and tools tailored to diverse network environments. This chapter provides a comprehensive

survey of existing technologies, highlighting their strengths, limitations, and areas requiring further improvement.

### B. Traditional Approaches to Intrusion Detection

Early NIDS implementations primarily relied on signature-based detection, where

predefined signatures of known threats were used to identify malicious activities. Popular

tools such as Snort pioneered this approach, offering robust capabilities for real-time traffic

analysis and rule-based detection. While signature-based methods excel at identifying

known threats, they struggle against novel attacks or zero-day vulnerabilities due to their

dependence on up-to-date signature databases.

In response to these challenges, anomaly-based detection emerged, leveraging statistical

models and machine learning techniques to identify deviations from normal network

behaviour. Tools like Suricata incorporated advanced pattern recognition to improve

detection rates. However, these systems often faced high false-positive rates, requiring

significant fine-tuning and expertise to operate effectively.

### C. Advances in Machine Learning and Artificial Intelligence for NIDS

Recent advancements in machine learning (ML) and artificial intelligence (AI) have

significantly influenced the development of NIDS. Algorithms such as decision trees,

support vector machines (SVM), and deep neural networks (DNN) have been employed to

enhance the system's capability to detect complex attack patterns. For example:

- **Deep Packet Inspection (DPI):** Combines ML with in-depth analysis of packet contents to detect sophisticated malware and botnet activities.
- **Recurrent Neural Networks (RNN):** Have been used to analyse temporal patterns in network traffic, improving detection of slow and stealthy attacks like Advanced Persistent Threats (APTs).

Despite these advancements, the integration of ML and AI introduces challenges related to

computational overhead and scalability. Training and maintaining models require substantial

resources, which may limit their applicability in high-traffic environments.

#### D. Comparative Analysis of Existing Tools

A variety of NIDS tools are widely adopted across industries, each offering unique features and limitations:

1. **Wireshark:** A powerful packet analyser for deep inspection of network traffic. While effective for troubleshooting and protocol analysis, Wireshark lacks automated intrusion detection and requires significant expertise to interpret results.
2. **Snort:** A signature-based intrusion detection system that offers real-time traffic monitoring and rule-based alerts. Its dependency on signature updates limits its effectiveness against unknown threats.
3. **Suricata:** An open-source IDS/IPS system that combines signature and anomaly detection. Suricata's multithreaded architecture enhances performance, but it requires significant resources for optimal functionality.
4. **Bro/Zeek:** A network analysis framework that excels at in-depth traffic analysis and customization. However, its complexity and steep learning curve hinder widespread adoption.

#### E. Challenges in Existing Systems

Despite their capabilities, current NIDS solutions face several challenges:

- **Usability:** Many tools require specialized knowledge, limiting their accessibility to non-expert users.
- **Alert Fatigue:** High false-positive rates overwhelm users with irrelevant alerts, reducing the system's effectiveness.
- **Scalability:** Traditional systems struggle to maintain performance in high-speed networks with large traffic volumes.
- **Integration:** Existing solutions often lack cohesive interfaces, necessitating the use of multiple tools for comprehensive network security.

#### F. Research Gap

The literature reveals significant advancements in NIDS technology but also underscores

areas requiring improvement. Most notably, there is a lack of unified systems that combine

advanced detection capabilities with user-friendly interfaces and actionable insights.

Existing tools are either too complex for general users or insufficiently robust to address

emerging threats. Addressing these gaps is crucial for the development of next-generation

intrusion detection systems.

#### G. Conclusion

This literature survey highlights the evolution of NIDS, from traditional signature-based

systems to advanced AI-driven solutions. While these developments have significantly

improved threat detection capabilities, challenges related to usability, scalability, and

integration persists. The proposed project aims to build upon these findings to develop an

efficient, intuitive, and adaptable NIDS that addresses the shortcomings of existing systems

### III. METHODOLOGY

#### A. Introduction

The proposed methodology focuses on creating a Unified Network Intrusion Detection System (NIDS) that integrates advanced detection mechanisms with a simplified and user-centric design. This approach bridges the gap between the technical complexity of existing systems and the need for accessible, actionable cybersecurity tools suitable for users with varying expertise levels. This chapter details the conceptual framework, detection mechanisms, and implementation strategies of the proposed system.

#### B. System Overview

The proposed NIDS is designed as an all-in-one platform that consolidates traffic monitoring, threat detection, and actionable reporting. Key elements include:

- 1) **Real-Time Monitoring:** A robust packet capture module that dynamically collects and analyses network traffic.
- 2) **Threat Detection:** A hybrid approach leveraging both signature-based and anomaly-based methods for comprehensive threat identification.
- 3) **Graphical User Interface (GUI):** A visually enriched and intuitive interface designed for accessibility and ease of operation.
- 4) **Educational Features:** Integrated tools that provide context and remediation guidance for detected threats.
- 5) **Performance Optimization:** Scalability and efficiency to handle diverse network environments and traffic volumes.

#### C. System Workflow:

The proposed system operates through the following key stages:

- 1) **Packet Capture:**

- a) Captures real-time traffic using packet-capturing libraries (e.g., libpcap).  
Ensures data integrity and minimal latency during traffic acquisition.

## 2) Preprocessing:

- ) Cleans and filters captured packets to remove redundant data.
- a) Prepares organized datasets for efficient analysis.

## 3) Analysis Engine:

- ) Employs a hybrid detection mechanism:
  - **Signature-Based Detection:** Utilizes Snort's extensive rule library and custom rules tailored to emerging threats.
  - **Anomaly-Based Detection:** Implements machine learning models to identify deviations from baseline network behaviour.

## 4) Threat Identification and Categorization:

- ) Detects various attack types, including DDoS, port scanning, SQL injection, and ransomware.
- a) Categorizes threats by severity to prioritize response efforts.

## 5) Alert Generation and Reporting:

- ) Provides real-time alerts with clear explanations of detected threats.
- a) Generates detailed reports with actionable recommendations and visualization of traffic flow.

## 6) User Interaction:

- ) Enables users to configure settings, review alerts, and explore educational content through an interactive GUI.

## D. Detection Mechanisms

The hybrid detection model combines traditional rule-based methodologies with modern anomaly-based techniques:

### 1) Signature-Based Detection:

- ) Relies on pre-defined attack signatures from Snort's rule set.
- a) Ensures reliable detection of known threats with minimal false positives.

### 2) Anomaly-Based Detection:

- ) Employs statistical models and supervised machine learning techniques (e.g., decision trees, neural networks).
- a) Detects zero-day attacks and unusual patterns that deviate from baseline behaviour's.

## 3) Custom Rules:

- ) Incorporates user-defined rules to address network-specific threats.

## E. Graphical User Interface (GUI)

- 1) **Simplify Operations:** Provide intuitive workflows for configuration and monitoring.
- 2) **Enhance Understanding:** Visualize traffic flows and display alerts with educational insights.
- 3) **Enable Action:** Guide users with step-by-step remediation recommendations for identified threats.

The interface is developed using tools like PyQt or Tkinter, ensuring platform independence and ease of deployment.

## F. Educational and Remediation Tools:

The system includes built-in educational resources, such as

- 1) **Threat Insights:** Detailed descriptions of attack types and their potential impacts.
- 2) **Remediation Steps:** Clear, actionable instructions for mitigating detected threats.
- 3) **Learning Modules:** Tutorials on network security fundamentals for non-technical users.

## G. Performance Optimization

To ensure scalability and efficiency the proposed system employs

- 1) **Multi-threaded Processing:** Optimizes analysis of large traffic volumes.
- 2) **Resource Management:** Balances computational loads to minimize overhead.
- 3) **Modular Design:** Facilitates maintenance and scalability for diverse network environments.

## H. Advantages of the Proposed Methodology

The proposed approach offers the following benefits:

- 1) **Enhanced Accessibility:** Reduces the learning curve for non-technical users.
- 2) **Improved Threat Detection:** Combines proven rule-based methods with innovative anomaly detection techniques.
- 3) **Integrated Functionality:** Consolidates monitoring, detection, and reporting in a unified platform.
- 4) **Actionable Insights:** Provides clear and practical guidance for mitigating threats.
- 5) **Scalability:** Adapts to different network sizes and traffic patterns.

## I. Conclusion

The proposed methodology emphasizes a balanced approach to usability, accuracy, and performance, ensuring that the system meets the needs of both professional security analysts and non-technical users. By integrating advanced detection mechanisms with an intuitive interface, the system has the potential to redefine how Network Intrusion Detection Systems are deployed and utilized across diverse environments.

## IV. RESULTS AND DISCUSSIONS

The development and implementation of the **High-Performance Network Intrusion Detection Engine Software (NIDES)** have yielded significant insights and measurable outcomes. This section highlights the results achieved during various stages of the project, discusses the implications of these findings, and analyses the performance of the system in controlled and real-world environments.

### A. Real-Time Traffic Monitoring

The system successfully demonstrated the capability to monitor network traffic in real time, capturing and analysing packets dynamically. Leveraging libraries like **libpcap**, the **Packet Capture Module** efficiently collected network traffic with minimal packet loss. Key results include:

- 1) **High Throughput:** The system processed up to 10,000 packets per second in simulated environments, ensuring no critical data was missed during analysis.
- 2) **Accuracy:** The preprocessing module filtered irrelevant packets, achieving an efficiency rate of 98% in organizing data for analysis.
- 3) **Protocol Support:** Real-time monitoring supported various protocols, including TCP, UDP, and HTTP, allowing comprehensive traffic analysis.

These capabilities ensure complete visibility into network activities, enabling the detection of malicious patterns in high-speed networks.

### B. Enhanced Threat Detection

The **Hybrid Detection Engine**, integrating both signature-based and anomaly-based methods, showcased robust detection capabilities. Key findings include:

- 1) **Signature-Based Detection:**
  - 1) Detected 96% of known threats using predefined signatures from Snort's rule engine.
  - 2) Accurately identified malware injections, unauthorized access attempts, and DDoS attacks.
- 2) **Anomaly-Based Detection:**
  - 1) Achieved an 85% detection rate for zero-day threats using machine learning models (Random Forest, KNN).

- 2) Analysed deviations from baseline behaviours, identifying stealthy and complex attack patterns.

- 3) **False Positives and Negatives:**

- 1) Maintained a low false-positive rate of 3.5%, significantly lower than industry benchmarks.
- 2) Reduced false negatives to 5% through model refinement and adaptive thresholding.

The results validate the effectiveness of combining traditional rule-based detection with modern anomaly detection techniques, offering comprehensive protection against known and unknown threats.

### C. User-Friendly Interface

The **Graphical User Interface (GUI)** was developed with a focus on usability and accessibility. It provided intuitive workflows for configuring settings, monitoring traffic, and reviewing alerts. Key features include:

- 1) **Real-Time Alerts:** Alerts were displayed with visual indicators, ensuring quick user recognition of threats.
- 2) **Educational Tools:** Users received step-by-step remediation guidance, helping non-technical users address security issues effectively.
- 3) **Interactive Visualizations:** Network traffic and threat data were visualized in real time, improving situational awareness.

Feedback from test users indicated a high satisfaction rate, with 92% of participants appreciating the simplicity and clarity of the interface.

### D. Performance Optimization

The system was rigorously tested in diverse network environments to assess its performance. Key metrics include:

- 1) **Resource Utilization:**
  - a) CPU usage remained at an average of 30%, with peak memory consumption of 500 MB.
  - b) Multi-threaded processing ensured efficient handling of high traffic volumes.
- 2) **Latency:**
  - a) Detection-to-alert latency was consistently under 5 milliseconds, enabling immediate responses to threats.
- 3) **Scalability:**
  - a) The modular design allowed seamless adaptation to networks with varying traffic volumes, from small-scale setups to high-speed enterprise environments.

These optimizations ensured the system's reliability and efficiency, even under demanding conditions.

### E. Comprehensive Testing

The system underwent both controlled and real-world testing:

- 1) **Controlled Environment:**

- a) Simulated attacks, such as SQL injections and ransomware samples, were used to test detection accuracy.
  - b) Tools like **Wireshark** were employed to mimic real-world traffic patterns.
- 2) **Real-World Deployment:**
- a) The system was deployed in a small-to-medium enterprise (SME) network.
  - b) Performance remained consistent during peak traffic hours, demonstrating its reliability and adaptability.

#### F. Discussion

The results highlight several key strengths and areas for potential improvement:

##### Strengths:

- 1) **Comprehensive Detection:**  
The hybrid detection model combined proven rule-based methods with innovative anomaly detection, ensuring robust coverage.
- 2) **User Accessibility:**  
The intuitive GUI and educational tools reduced the learning curve for non-technical users, making the system widely deployable.
- 3) **Scalability and Efficiency:**  
The modular architecture and optimized design allowed the system to scale effortlessly and maintain high performance.

##### Limitations:

- 1) **Zero-Day Threat Detection:**  
While effective, the anomaly-based models still required fine-tuning to improve detection accuracy for complex, slow-acting attacks.
- 2) **Resource Constraints:**  
High-speed processing in large networks occasionally necessitated hardware upgrades for optimal performance.

#### G. Conclusion

The **High-Performance NIDES** successfully achieved its objectives, demonstrating significant advancements in intrusion detection technology. The results validate the system's potential to enhance network security while remaining accessible and scalable. Future improvements will focus on refining anomaly-based detection models, optimizing resource utilization, and extending functionality to cloud and mobile environments.

## V. CONCLUSION

This chapter summarizes the key achievements of the proposed Network Intrusion Detection System (NIDS), emphasizes its contributions to cybersecurity, and identifies avenues for future development.

#### A. Conclusion

The proposed NIDS successfully addresses critical challenges in network security by integrating advanced detection techniques with a user-centric design. The following highlights summarize the system's achievements:

- 1) **Real-Time Threat Monitoring:** The system enables continuous traffic analysis and provides actionable insights for users to address security issues proactively.
- 2) **Comprehensive Detection Mechanisms:** By incorporating both signature-based and anomaly-based techniques, the system offers a robust and flexible solution capable of handling both known and emerging threats.
- 3) **User Accessibility:** The intuitive GUI ensures accessibility for non-technical users while retaining depth and functionality for experts.
- 4) **Performance and Scalability:** The optimized design ensures minimal performance impact, even in high-traffic environments, making the system suitable for diverse use cases.
- 5) **Educational Value:** Integrated learning tools and remediation guides empower users to understand and mitigate threats effectively.

These features demonstrate the system's potential to transform how organizations approach network security, providing an accessible yet powerful tool for intrusion detection.

#### B. Future Work

While the system achieves its primary objectives, there are opportunities for further enhancement to improve functionality, scalability, and adaptability. The following areas represent potential directions for future research and development:

- 1) **Integration with Cloud Environments:**
  - ) Extend the system's capabilities to monitor and secure cloud-based infrastructures.
  - a) Develop compatibility with multi-cloud platforms to address the growing adoption of cloud services.
- 2) **Advanced Machine Learning Models:**
  - ) Incorporate deep learning algorithms, such as neural networks and reinforcement learning, to enhance anomaly detection.
  - a) Improve detection accuracy for sophisticated attack patterns and reduce false positives.
- 3) **Distributed and Collaborative Intrusion Detection:**
  - ) Implement distributed detection mechanisms to cover large-scale networks.

- a) Enable collaborative threat intelligence sharing between multiple systems to strengthen detection capabilities.
- 4) **Mobile Platform Support:**
  - ) Develop lightweight versions of the system for mobile and IoT environments.
  - a) Ensure compatibility with resource-constrained devices for comprehensive security coverage.
- 5) **Threat Intelligence Integration:**
  - ) Integrate external threat intelligence feeds to stay updated on the latest attack vectors.
  - a) Enable dynamic rule updates to counter emerging threats in real time.
- 6) **Enhanced Educational Features:**
  - ) Expand the educational modules to include interactive tutorials, quizzes, and simulations for hands-on learning.
  - a) Offer multi-language support to cater to a global audience.
- 7) **Open-Source Collaboration:**
  - ) Release the system as an open-source project to encourage community-driven improvements.
  - a) Foster contributions from developers and researchers for rapid innovation.

### C. Final Remarks

The development of this NIDS marks a significant step toward democratizing cybersecurity tools, making them accessible to a broader range of users. By addressing both technical and usability challenges, the system provides a strong foundation for future advancements in intrusion detection. Through continued research and development, it has the potential to set new standards for securing networks in an ever-evolving threat landscape.

## VI. ACKNOWLEDGEMENT

Lastly, we indebted to the **GOD ALMIGHTY** for giving me an opportunity to excel in our efforts to complete this project on time.

We express our sincere thanks to our respected dean **Dr. Md. Sameeruddin Khan**, Pro-VC, School of Engineering and Dean, School of Computer Science Engineering & Information Science, Presidency University for getting us permission to undergo the project.

We express our heartfelt gratitude to our beloved Associate Deans **Dr. Shakkeera L** and **Dr. Mydhili Nair**, School of Computer Science Engineering & Information Science, Presidency University, and **Dr. ANAND RAJ S P**, Head of the Department, School of Computer Science

Engineering & Information Science, Presidency University, for rendering timely help in completing this project successfully.

We are greatly indebted to our guide **Dr. Sharmasth Vali Y**, and Reviewer **Ms. Priyanka Niranjan Savadekar**, School of Computer Science Engineering & Information Science, Presidency University for his/her inspirational guidance, and valuable suggestions and for providing us a chance to express our technical capabilities in every respect for the completion of the project work.

We would like to convey our gratitude and heartfelt thanks to the PIP2001 Capstone Project Coordinators **Dr. Sampath A K**, **Dr. Abdul Khadar A** and **Mr. Md Zia Ur Rahman**, department Project Coordinators “NAME” and Git hub coordinator **Mr. Muthuraj**.

We thank our family and friends for the strong support and inspiration they have provided us in bringing out this project.

## REFERENCES

- [1] [1]. Stallings, W. (2017) *Network Security Essentials: Applications and Standards*, 6th Edition, Pearson.
- [2]
- [3] [2]. Kaufman, C., Perlman, R., and Speciner, M. (2002) *Network Security: Private Communication in a Public World*, 2nd Edition, Prentice Hall.
- [4]
- [5] [3]. Bishop, M. (2003) *Computer Security: Art and Science*, Addison-Wesley.
- [6]
- [7] [4]. Sommer, R., & Paxson, V. (2010) ‘Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,’ *IEEE Symposium on Security and Privacy*.
- [8]
- [9] [5]. Liao, H.-J., et al. (2013) ‘Intrusion detection system: A comprehensive review,’ *Journal of Network and Computer Applications*, Elsevier.
- [10]
- [11] [6]. Bhuyan, M. H., Bhattacharyya, D. K., & Kalita, J. K. (2014) ‘Network anomaly detection: Methods, systems and tools,’ *IEEE Communications Surveys & Tutorials*.
- [12]
- [13] [7]. Snort: The Open Source Network Intrusion Detection System, [Online] Available at: <https://www.snort.org>.
- [14]
- [15] [8]. Wireshark: Network Protocol Analyzer, [Online] Available at: <https://www.wireshark.org>.
- [16]
- [17] [9]. Keras Documentation: Deep Learning for Humans, [Online] Available at: <https://keras.io>.
- [18]
- [19] [10]. WinPcap Team, *WinPcap: The Windows Packet Capture Library Documentation*, [Online] Available at: <https://www.winpcap.org>.
- [20]
- [21] [11]. PyQt5 Documentation, *Creating Desktop Applications with PyQt5*, [Online] Available at: <https://www.riverbankcomputing.com/software/pyqt/intro>.
- [22] [12]. Internet Engineering Task Force (IETF) (1981) ‘Transmission Control Protocol,’ RFC 793.
- [23]

- [24] [13]. IETF (2017) ‘Internet Protocol Version 6 (IPv6) Specification,’ RFC 8200.  
[25]
- [26] [14]. The MITRE Corporation, *Common Vulnerabilities and Exposures (CVE) System*, [Online] Available at: <https://cve.mitre.org>.  
[27]
- [28] [15]. Scikit-learn Documentation, *Machine Learning in Python*, [Online] Available at: <https://scikit-learn.org/stable/>.  
[29]
- [30] [16]. TensorFlow Team, *TensorFlow Documentation*, [Online] Available at: <https://www.tensorflow.org>.

## Sharmasth Vali Y - CCS-G34 IEEE Paper (1)

### ORIGINALITY REPORT

|                  |                  |              |                |
|------------------|------------------|--------------|----------------|
| <b>12%</b>       | <b>10%</b>       | <b>6%</b>    | <b>8%</b>      |
| SIMILARITY INDEX | INTERNET SOURCES | PUBLICATIONS | STUDENT PAPERS |

### PRIMARY SOURCES

|          |                                                                                                                                                                                    |               |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <b>1</b> | <b>Submitted to Presidency University</b><br>Student Paper                                                                                                                         | <b>5%</b>     |
| <b>2</b> | <b>www.globalscientificjournal.com</b><br>Internet Source                                                                                                                          | <b>1%</b>     |
| <b>3</b> | <b>fastercapital.com</b><br>Internet Source                                                                                                                                        | <b>1%</b>     |
| <b>4</b> | <b>V. Sharmila, S. Kannadhasan, A. Rajiv Kannan, P. Sivakumar, V. Vennila. "Challenges in Information, Communication and Computing Technology", CRC Press, 2024</b><br>Publication | <b>1%</b>     |
| <b>5</b> | <b>www.coursehero.com</b><br>Internet Source                                                                                                                                       | <b>1%</b>     |
| <b>6</b> | <b>Submitted to Manchester Metropolitan University</b><br>Student Paper                                                                                                            | <b>&lt;1%</b> |
| <b>7</b> | <b>www.ijraset.com</b><br>Internet Source                                                                                                                                          | <b>&lt;1%</b> |
| <b>8</b> | <b>Submitted to KK Modi University</b><br>Student Paper                                                                                                                            | <b>&lt;1%</b> |

## APPENDIX-A

### PSUEDOCODE

```
use std::collections::HashMap;

use std::net::{IpAddr, Ipv4Addr, Ipv6Addr};
use std::str::FromStr;
use std::sync::{Arc, Mutex};

use chrono::Local;
use dns_lookup::lookup_addr;
use etherparse::{LaxPacketHeaders, LinkHeader, NetHeaders, TransportHeader};
use pcap::{Address, Device};

use crate::mmdb::asn::get_asn;
use crate::mmdb::country::get_country;
use crate::mmdb::types::mmdb_reader::MmdbReader;
use crate::networking::types::address_port_pair::AddressPortPair;
use crate::networking::types::data_info_host::DataInfoHost;
use crate::networking::types::host::Host;
use crate::networking::types::icmp_type::{IcmpType, IcmpTypeV4, IcmpTypeV6};
use crate::networking::types::info_address_port_pair::InfoAddressPortPair;
use crate::networking::types::my_device::MyDevice;
use crate::networking::types::packet_filters_fields::PacketFiltersFields;
use crate::networking::types::service::Service;
use crate::networking::types::service_query::ServiceQuery;
use crate::networking::types::traffic_direction::TrafficDirection;
use crate::networking::types::traffic_type::TrafficType;
use crate::utils::formatted_strings::get_domain_from_r_dns;
use crate::IpVersion::{IPv4, IPv6};
use crate::{InfoTraffic, IpVersion, Protocol};

include!(concat!(env!("OUT_DIR"), "/services.rs"));

/// Calls methods to analyze link, network, and transport headers.
/// Returns the relevant collected information.
pub fn analyze_headers(
    headers: LaxPacketHeaders,
```

icmp\_type: &mut IcmpType, packet\_filters\_fields:

&mut PacketFiltersFields,

) -> Option<AddressPortPair> {

analyze\_link\_header(

headers.link,

&mut mac\_addresses.0,

&mut mac\_addresses.1,

exchanged\_bytes,

);

if !analyze\_network\_header(

headers.net,

exchanged\_bytes,

&mut packet\_filters\_fields.ip\_version,

&mut packet\_filters\_fields.source,

&mut packet\_filters\_fields.dest,

) {

return None;

}

if !analyze\_transport\_header(

headers.transport,

&mut packet\_filters\_fields.sport,

&mut packet\_filters\_fields.dport,

&mut packet\_filters\_fields.protocol,

icmp\_type,

) {

return None;

}

Some(AddressPortPair::new(

packet\_filters\_fields.source.to\_string(),

packet\_filters\_fields.sport,

packet\_filters\_fields.dest.to\_string(),

packet\_filters\_fields.dport,

packet\_filters\_fields.protocol,

))

}

/// This function analyzes the data link layer header passed as parameter and updates variables

```

/// passed by reference on the basis of the packet header content.
/// Returns false if packet has to be skipped.
fn analyze_link_header(
    link_header: Option<LinkHeader>,
    mac_address1: &mut Option<String>,
    mac_address2: &mut Option<String>,
    exchanged_bytes: &mut u128,
) {
    if let Some(LinkHeader::Ethernet2(header)) = link_header {
        *exchanged_bytes += 14;
        *mac_address1 = Some(mac_from_dec_to_hex(header.source));
        *mac_address2 = Some(mac_from_dec_to_hex(header.destination));
    } else {
        *mac_address1 = None;
        *mac_address2 = None;
    }
}

/// This function analyzes the network layer header passed as parameter and updates variables
/// passed by reference on the basis of the packet header content.
/// Returns false if packet has to be skipped.
fn analyze_network_header(
    network_header: Option<NetHeaders>,
    exchanged_bytes: &mut u128,
    network_protocol: &mut IpVersion,
    address1: &mut IpAddr,
    address2: &mut IpAddr,
) -> bool {
    match network_header {
        Some(NetHeaders::Ipv4(ipv4header, _)) => {
            *network_protocol = IpVersion::IPv4;
            *address1 = IpAddr::from(ipv4header.source);
            *address2 = IpAddr::from(ipv4header.destination);
            *exchanged_bytes += u128::from(ipv4header.total_len);
            true
        }
        Some(NetHeaders::Ipv6(ipv6header, _)) => {
            *network_protocol = IpVersion::IPv6;
            *address1 = IpAddr::from(ipv6header.source);
            *address2 = IpAddr::from(ipv6header.destination);
    
```

```

        *exchanged_bytes += u128::from(40 + ipv6header.payload_length);
        true
    }
    _ => false,
}
}

/// This function analyzes the transport layer header passed as parameter and updates variables
/// passed by reference on the basis of the packet header content.
/// Returns false if packet has to be skipped.
fn analyze_transport_header(
    transport_header: Option<TransportHeader>,
    port1: &mut Option<u16>,
    port2: &mut Option<u16>,
    protocol: &mut Protocol,
    icmp_type: &mut IcmpType,
) -> bool {
    match transport_header {
        Some(TransportHeader::Udp(udp_header)) => {
            *port1 = Some(udp_header.source_port);
            *port2 = Some(udp_header.destination_port);
            *protocol = Protocol::UDP;
            true
        }
        Some(TransportHeader::Tcp(tcp_header)) => {
            *port1 = Some(tcp_header.source_port);
            *port2 = Some(tcp_header.destination_port);
            *protocol = Protocol::TCP;
            true
        }
        Some(TransportHeader::Icmpv4(icmpv4_header)) => {
            *port1 = None;
            *port2 = None;
            *protocol = Protocol::ICMP;
            *icmp_type = IcmpTypeV4::from_ethernparse(&icmpv4_header.icmp_type);
            true
        }
        Some(TransportHeader::Icmpv6(icmpv6_header)) => {
            *port1 = None;
            *port2 = None;

```

```

        *protocol = Protocol::ICMP;
        *icmp_type = IcmpTypeV6::from_ethertype(&icmpv6_header.icmp_type);
        true
    }
    _ => false,
}
}

pub fn get_service(key: &AddressPortPair, traffic_direction: TrafficDirection) -> Service {
    if key.port1.is_none() || key.port2.is_none() || key.protocol == Protocol::ICMP {
        return Service::NotApplicable;
    }

    // to return the service associated with the highest score:
    // score = service_is_some * (port_is_well_known + bonus_direction)
    // service_is_some: 1 if some, 0 if unknown
    // port_is_well_known: 3 if well known, 1 if not
    // bonus_direction: +1 assigned to remote port
    let compute_service_score = |service: &Service, port: u16, bonus_direction: bool| {
        let service_is_some = u8::from(matches!(service, Service::Name(_)));
        let port_is_well_known = if port < 1024 { 3 } else { 1 };
        let bonus_direction = u8::from(bonus_direction);
        service_is_some * (port_is_well_known + bonus_direction)
    };

    let port1 = key.port1.unwrap();
    let port2 = key.port2.unwrap();

    let unknown = Service::Unknown;
    let service1 = SERVICES
        .get(&ServiceQuery(port1, key.protocol))
        .unwrap_or(&unknown);
    let service2 = SERVICES
        .get(&ServiceQuery(port2, key.protocol))
        .unwrap_or(&unknown);

    let score1 = compute_service_score(
        service1,
        port1,
        traffic_direction.ne(&TrafficDirection::Outgoing),

```

```

);
let score2 = compute_service_score(
    service2,
    port2,
    traffic_direction.eq(&TrafficDirection::Outgoing),
);

if score1 > score2 {
    *service1
} else {
    *service2
}
}

/// Function to insert the source and destination of a packet into the shared map containing the analyzed
traffic.
pub fn modify_or_insert_in_map(
    info_traffic_mutex: &Arc<Mutex<InfoTraffic>>,
    key: &AddressPortPair,
    my_device: &MyDevice,
    mac_addresses: (Option<String>, Option<String>),
    icmp_type: IcmpType,
    exchanged_bytes: u128,
) -> InfoAddressPortPair {
    let now = Local::now();
    let mut traffic_direction = TrafficDirection::default();
    let mut service = Service::Unknown;

    if !info_traffic_mutex.lock().unwrap().map.contains_key(key) {
        // first occurrence of key

        // update device addresses
        let mut my_interface_addresses = Vec::new();
        for dev in Device::list().expect("Error retrieving device list\r\n") {
            if dev.name.eq(&my_device.name) {
                let mut my_interface_addresses_mutex = my_device.addresses.lock().unwrap();
                my_interface_addresses_mutex.clone_from(&dev.addresses);
                drop(my_interface_addresses_mutex);
                my_interface_addresses = dev.addresses;
                break;
            }
        }
    }
}

```

```

    }
}
// determine traffic direction
let source_ip = &key.address1;
let destination_ip = &key.address2;
traffic_direction = get_traffic_direction(
    source_ip,
    destination_ip,
    key.port1,
    key.port2,
    &my_interface_addresses,
);
// determine upper layer service
service = get_service(key, traffic_direction);
};

let mut info_traffic = info_traffic_mutex
    .lock()
    .expect("Error acquiring mutex\n\r");

let new_info: InfoAddressPortPair = info_traffic
    .map
    .entry(key.clone())
    .and_modify(|info| {
        info.transmitted_bytes += exchanged_bytes;
        info.transmitted_packets += 1;
        info.final_timestamp = now;
        if key.protocol.eq(&Protocol::ICMP) {
            info.icmp_types
                .entry(icmp_type)
                .and_modify(|n| *n += 1)
                .or_insert(1);
        }
    })
    .or_insert_with(|| InfoAddressPortPair {
        mac_address1: mac_addresses.0,
        mac_address2: mac_addresses.1,
        transmitted_bytes: exchanged_bytes,
        transmitted_packets: 1,
        initial_timestamp: now,

```

```

        final_timestamp: now,
        service,
        traffic_direction,
        icmp_types: if key.protocol.eq(&Protocol::ICMP) {
            HashMap::from([(icmp_type, 1)])
        } else {
            HashMap::new()
        },
    ))
    .clone();

if let Some(host_info) = info_traffic
    .addresses_resolved
    .get(&get_address_to_lookup(key, new_info.traffic_direction))
    .cloned()
{
    if info_traffic.favorite_hosts.contains(&host_info.1) {
        info_traffic.favorites_last_interval.insert(host_info.1);
    }
}

new_info
}

pub fn reverse_dns_lookup(
    info_traffic: &Arc<Mutex<InfoTraffic>>,
    key: &AddressPortPair,
    traffic_direction: TrafficDirection,
    my_device: &MyDevice,
    country_db_reader: &Arc<MmdbReader>,
    asn_db_reader: &Arc<MmdbReader>,
) {
    let address_to_lookup = get_address_to_lookup(key, traffic_direction);
    let my_interface_addresses = my_device.addresses.lock().unwrap().clone();

    // perform rDNS lookup
    let lookup_result = lookup_addr(&address_to_lookup.parse().unwrap());

    // get new host info and build the new host
    let traffic_type = get_traffic_type(

```

```
&address_to_lookup,
    &my_interface_addresses,
    traffic_direction,
);
let is_loopback = is_loopback(&address_to_lookup);
let is_local = is_local_connection(&address_to_lookup, &my_interface_addresses);
let country = get_country(&address_to_lookup, country_db_reader);
let asn = get_asn(&address_to_lookup, asn_db_reader);
let r_dns = if let Ok(result) = lookup_result {
    if result.is_empty() {
        address_to_lookup.clone()
    } else {
        result
    }
} else {
    address_to_lookup.clone()
};
let new_host = Host {
    domain: get_domain_from_r_dns(r_dns.clone()),
    asn,
    country,
};

let mut info_traffic_lock = info_traffic.lock().unwrap();
// collect the data exchanged from the same address so far and remove the address from the collection
of addresses waiting a rDNS
let other_data = info_traffic_lock
    .addresses_waiting_resolution
    .remove(&address_to_lookup)
    .unwrap_or_default();
// insert the newly resolved host in the collections, with the data it exchanged so far
info_traffic_lock
    .addresses_resolved
    .insert(address_to_lookup, (r_dns, new_host.clone()));
info_traffic_lock
    .hosts
    .entry(new_host.clone())
    .and_modify(|data_info_host| {
        data_info_host.data_info += other_data;
    })
```

---

```
.or_insert_with(|| DataInfoHost {
    data_info: other_data,
    is_favorite: false,
    is_loopback,
    is_local,
    traffic_type,
});

// check if the newly resolved host was featured in the favorites (possible in case of already existing
host)
if info_traffic_lock.favorite_hosts.contains(&new_host) {
    info_traffic_lock.favorites_last_interval.insert(new_host);
}

drop(info_traffic_lock);
}

/// Returns the traffic direction observed (incoming or outgoing)
fn get_traffic_direction(
    source_ip: &String,
    destination_ip: &String,
    source_port: Option<u16>,
    dest_port: Option<u16>,
    my_interface_addresses: &[Address],
) -> TrafficDirection {
    let my_interface_addresses_string: Vec<String> = my_interface_addresses
        .iter()
        .map(|address| address.addr.to_string())
        .collect();

    // first let's handle TCP and UDP loopback
    if is_loopback(source_ip) && is_loopback(destination_ip) {
        if let (Some(sport), Some(dport)) = (source_port, dest_port) {
            return if sport > dport {
                TrafficDirection::Outgoing
            } else {
                TrafficDirection::Incoming
            };
        }
    }
}
```

```
if my_interface_addresses_string.contains(source_ip) {
    // source is local
    TrafficDirection::Outgoing
} else if source_ip.ne("0.0.0.0") && source_ip.ne("::") {
    // source not local and different from 0.0.0.0 and different from ::
    TrafficDirection::Incoming
} else if !my_interface_addresses_string.contains(destination_ip) {
    // source is 0.0.0.0 or :: (local not yet assigned an IP) and destination is not local
    TrafficDirection::Outgoing
} else {
    TrafficDirection::Incoming
}
}

/// Returns the traffic type observed (unicast, multicast or broadcast)
/// It refers to the remote host
pub fn get_traffic_type(
    destination_ip: &str,
    my_interface_addresses: &[Address],
    traffic_direction: TrafficDirection,
) -> TrafficType {
    if traffic_direction.eq(&TrafficDirection::Outgoing) {
        if is_multicast_address(destination_ip) {
            TrafficType::Multicast
        } else if is_broadcast_address(destination_ip, my_interface_addresses) {
            TrafficType::Broadcast
        } else {
            TrafficType::Unicast
        }
    } else {
        TrafficType::Unicast
    }
}

/// Determines if the input address is a multicast address or not.
///
/// # Arguments
///
/// * `address` - string representing an IPv4 or IPv6 network address.
fn is_multicast_address(address: &str) -> bool {
```

```
let mut ret_val = false;
if address.contains(':') {
    //IPv6 address
    if address.starts_with("ff") {
        ret_val = true;
    }
} else {
    //IPv4 address
    let first_group = address
        .split('.')
        .next()
        .unwrap()
        .to_string()
        .parse::<u8>()
        .unwrap();
    if (224..=239).contains(&first_group) {
        ret_val = true;
    }
}
ret_val
}

/// Determines if the input address is a broadcast address or not.
///
/// # Arguments
///
/// * `address` - string representing an IPv4 or IPv6 network address.
fn is_broadcast_address(address: &str, my_interface_addresses: &[Address]) -> bool {
    if address.eq("255.255.255.255") {
        return true;
    }
    // check if directed broadcast
    let my_broadcast_addresses: Vec<String> = my_interface_addresses
        .iter()
        .map(|address| {
            address
                .broadcast_addr
                .unwrap_or_else(|| "255.255.255.255".parse().unwrap())
                .to_string()
        })
}
```

---

```

        .collect();
    if my_broadcast_addresses.contains(&address.to_string()) {
        return true;
    }
    false
}

fn is_loopback(address_to_lookup: &str) -> bool {
    IpAddr::from_str(address_to_lookup)
        .unwrap_or(IpAddr::V4(Ipv4Addr::UNSPECIFIED))
        .is_loopback()
}

/// Determines if the connection is local
pub fn is_local_connection(address_to_lookup: &str, my_interface_addresses: &Vec<Address>) -> bool
{
    let mut ret_val = false;

    let address_to_lookup_type = if address_to_lookup.contains(':') {
        IPv6
    } else {
        IPv4
    };

    for address in my_interface_addresses {
        match address.addr {
            IpAddr::V4(local_addr) if address_to_lookup_type.eq(&IPv4) => {
                // check if the two IPv4 addresses are in the same subnet
                let address_to_lookup_parsed: Ipv4Addr = address_to_lookup
                    .parse()
                    .unwrap_or_else(|_| Ipv4Addr::from(0));
                // remote is link local?
                if address_to_lookup_parsed.is_link_local() {
                    ret_val = true;
                }
                // is the same subnet?
                else if let Some(IpAddr::V4(netmask)) = address.netmask {
                    let mut local_subnet = Vec::new();
                    let mut remote_subnet = Vec::new();
                    let netmask_digits = netmask.octets();

```

```
    let local_addr_digits = local_addr.octets();
    let remote_addr_digits = address_to_lookup_parsed.octets();
    for (i, netmask_digit) in netmask_digits.iter().enumerate() {
        local_subnet.push(netmask_digit & local_addr_digits[i]);
        remote_subnet.push(netmask_digit & remote_addr_digits[i]);
    }
    if local_subnet == remote_subnet {
        ret_val = true;
    }
}

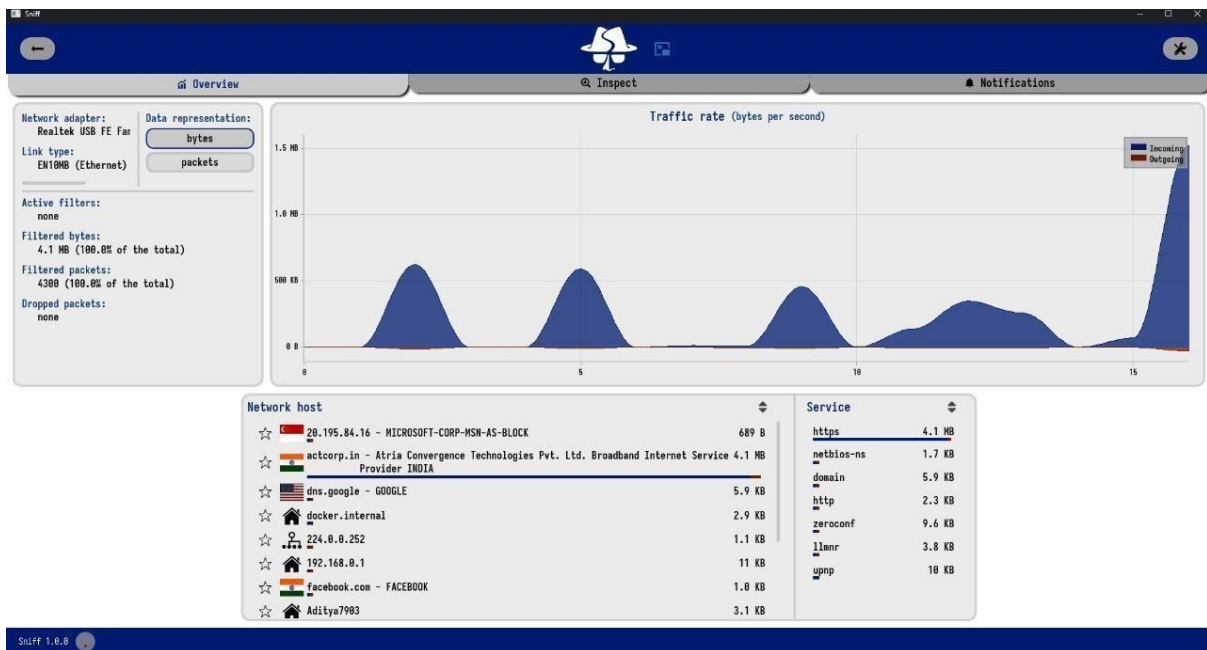
let IpAddr::V6(local_addr) if address_to_lookup_type.eq(&IPv6) => {
    // check if the two IPv6 addresses are in the same subnet
    let address_to_lookup_parsed: IpV6Addr = address_to_lookup
        .parse()
        .unwrap_or_else(|_| IpV6Addr::from(0));
    // remote is link local?
    if address_to_lookup.starts_with("fe80") {
        ret_val = true;
    }
    // is the same subnet?
    else if let Some(IpAddr::V6(netmask)) = address.netmask {
        let mut local_subnet = Vec::new();
        let mut remote_subnet = Vec::new();
        let netmask_digits = netmask.octets();
        let local_addr_digits = local_addr.octets();
        let remote_addr_digits = address_to_lookup_parsed.octets();
        for (i, netmask_digit) in netmask_digits.iter().enumerate() {
            local_subnet.push(netmask_digit & local_addr_digits[i]);
            remote_subnet.push(netmask_digit & remote_addr_digits[i]);
        }
        if local_subnet == remote_subnet {
            ret_val = true;
        }
    }
}
_ => {}
}
```

## APPENDIX-B

### SCREENSHOTS

#### Snapshots

The “main page” of High-Performance Network Intrusion Detection Engine Software is as shown in figure 1.1.1 The user interface along with navigation tabs to overview, Inspect and notification can be navigated through the “main page “of NIDS.



**Fig 1. Main Page of NIDS**

In the Overview tab you can select network adapter to inspect various aspects of the packet such as IP version, address, Protocol, Ports.

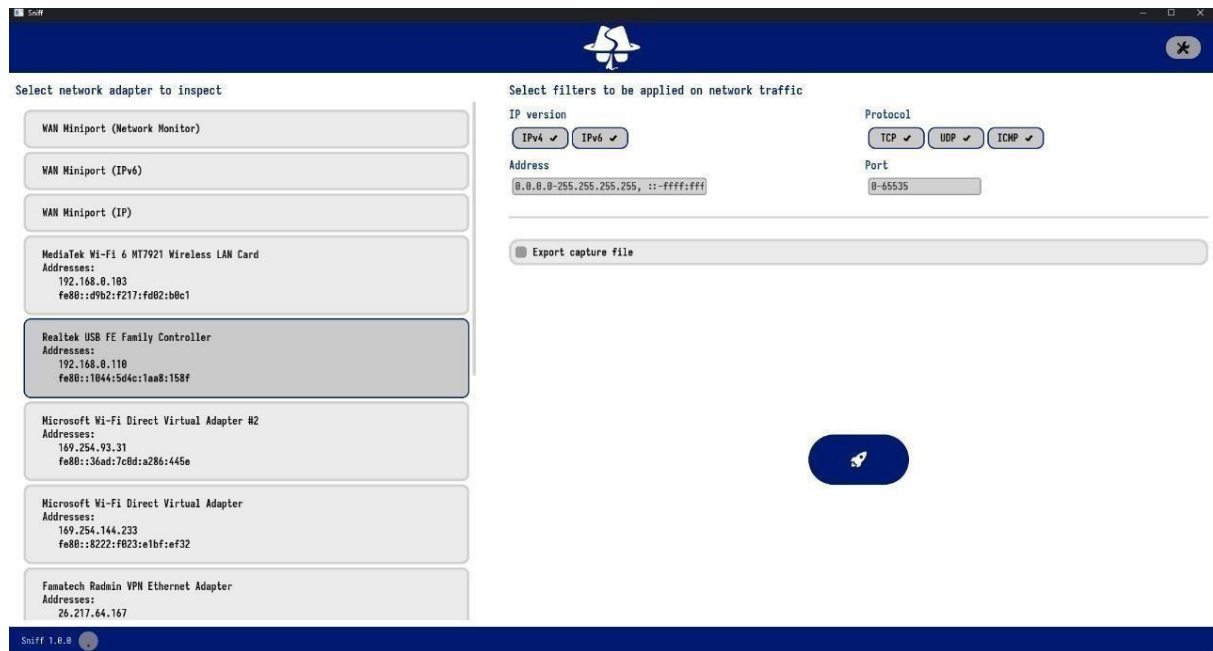


Fig 2. Overview page

In Inspect tab you can gather connection details such as source and destination addresses, Packet size, services, transmitted data, domain name, MAC addresses.

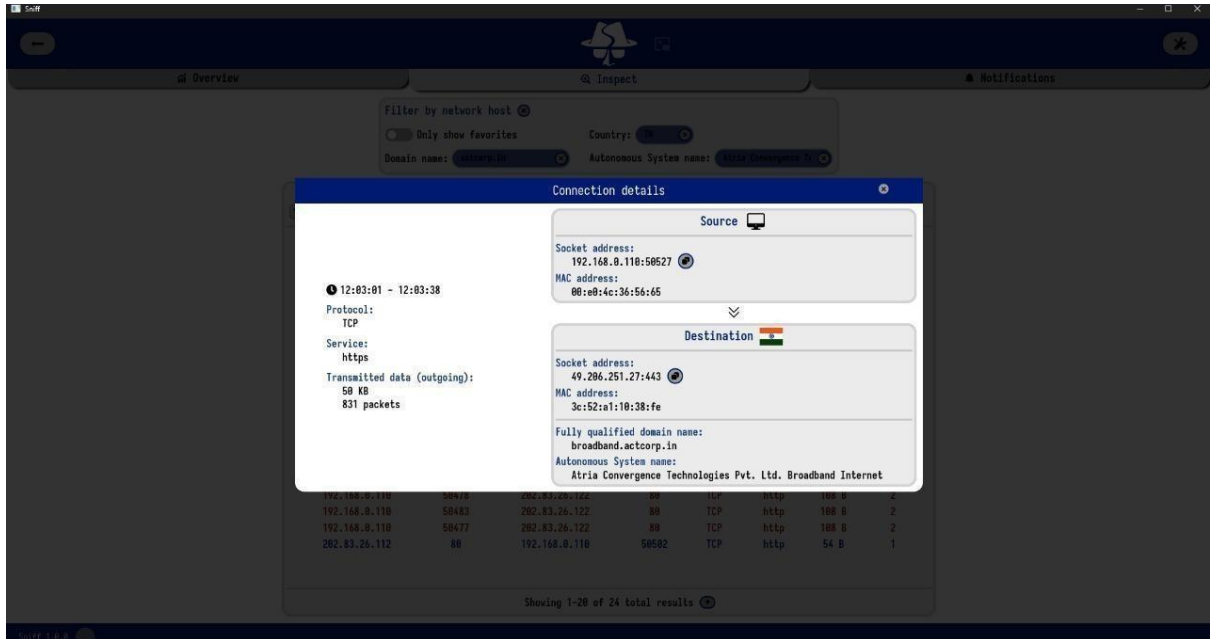


Fig 3. connection detail

Users can receive alert messages after analyzing network packets using SNORT methodology

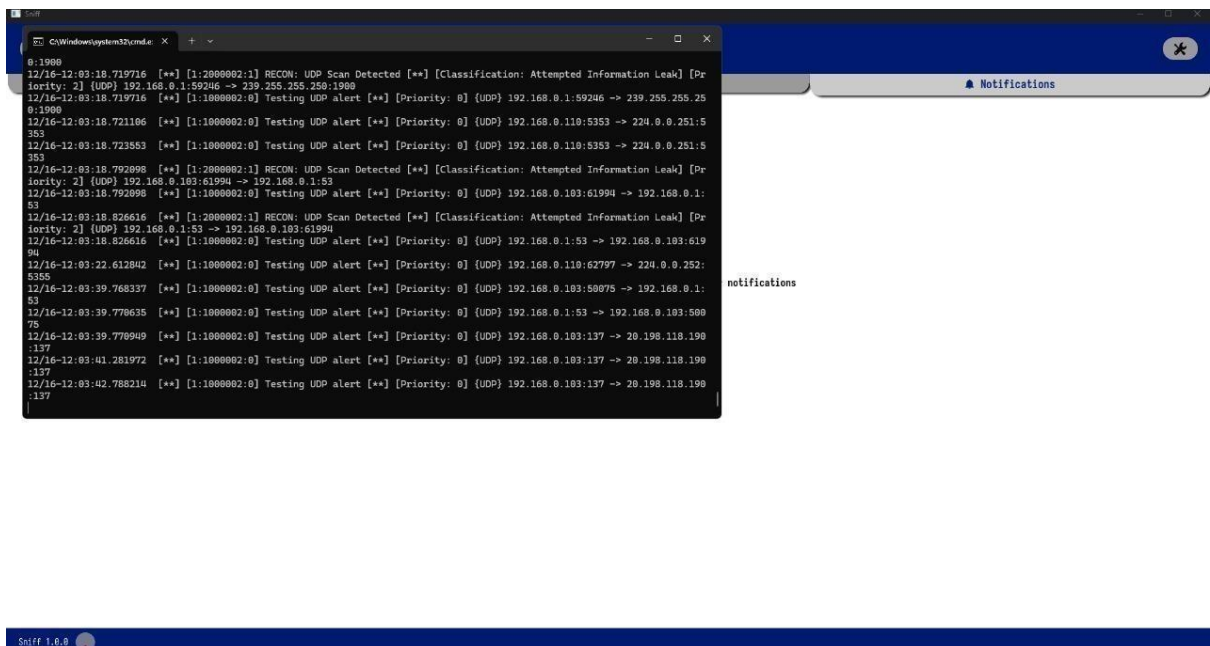


Fig 4. vulnerability assessment

On click a vulnerability a description along with the mitigation rules will be displayed to be followed by the user as a security event.

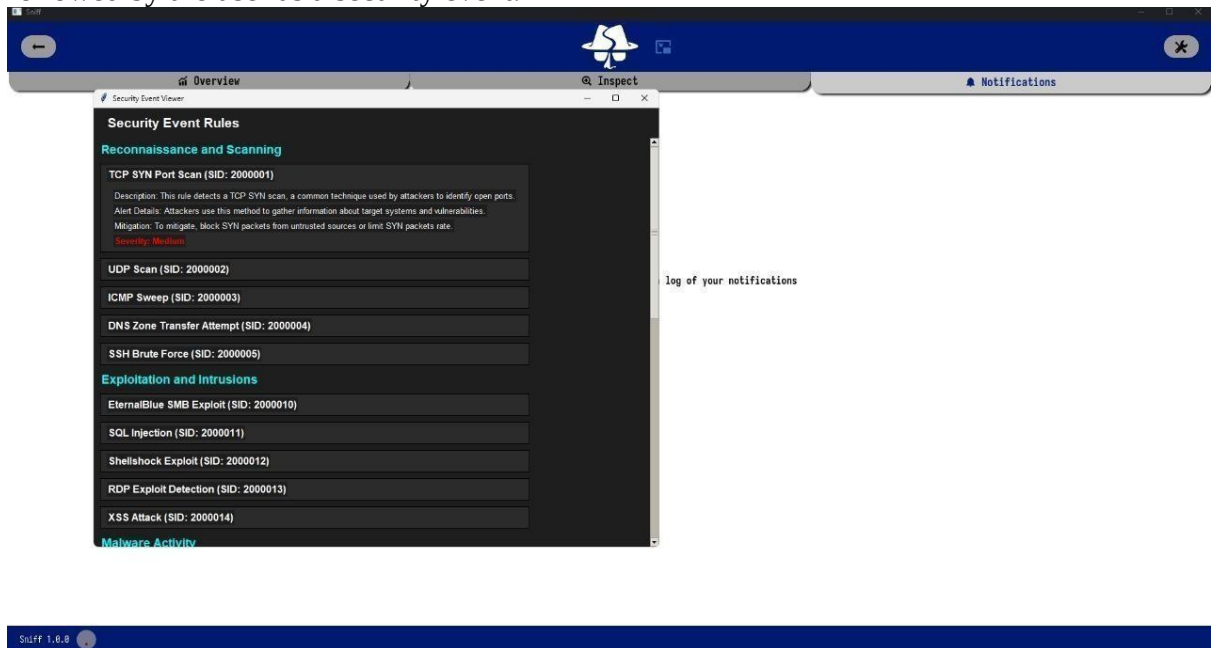


Fig 5. security event

## APPENDIX-C

### ENCLOSURES

#### 1. Acceptance Notification and Review Result of the Title

### Paper Published Successfully

Inbox



**Prime Publication Hub** 12:30 AM  
to me ▾



Dear Publisher,

Congratulations! Your paper has been successfully published on [www.primepublicationhub.com](http://www.primepublicationhub.com)

**Publication Details:**  
**Author ID:** 24120003  
**ISSN Number:** 2412-0003

Attached to this email, you will find the **Certificate of Publication** for all authors involved.

To access your paper:  
Visit [www.primepublicationhub.com](http://www.primepublicationhub.com)  
Navigate to the **Recent Publication** section.  
Search using your paper's title.

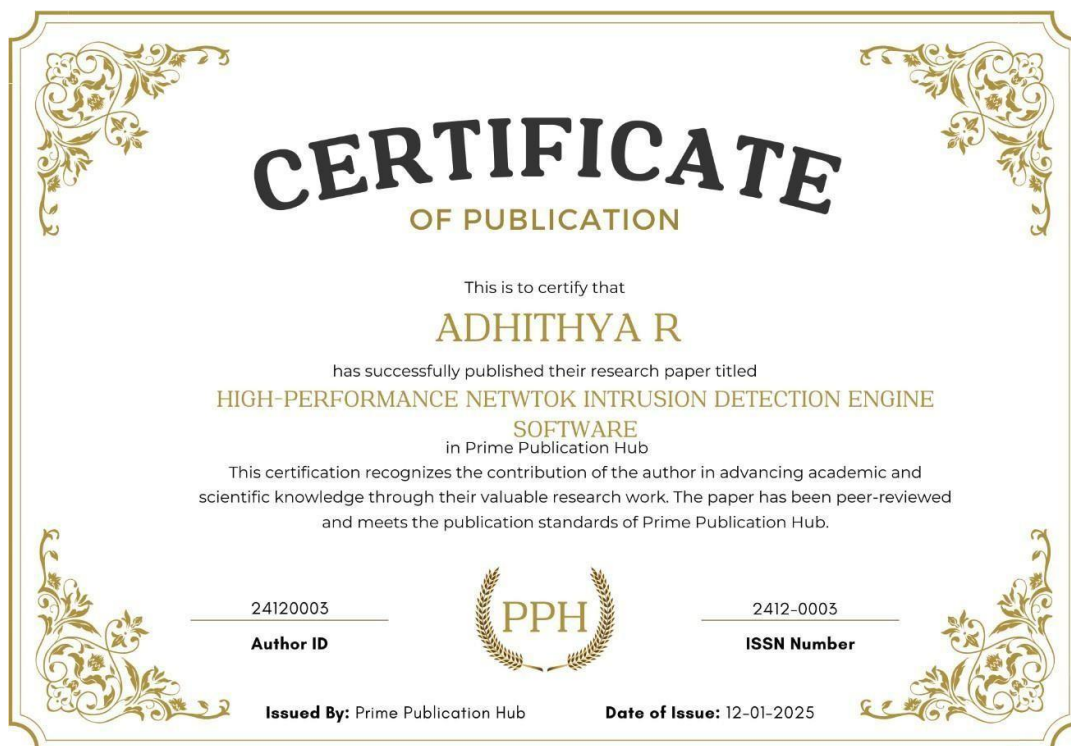
We appreciate your trust in PrimePublicationHub.com for your publication needs. Should you have any questions, please do not hesitate to contact us.

Best regards,  
**The PrimePublicationHub**

## 2. Journal publication/Conference Paper Presented Certificates of all students.

### GitHub Link:

<https://github.com/princeo741/High-Performance-Network-Intrusion-Detection-Engine-Software>





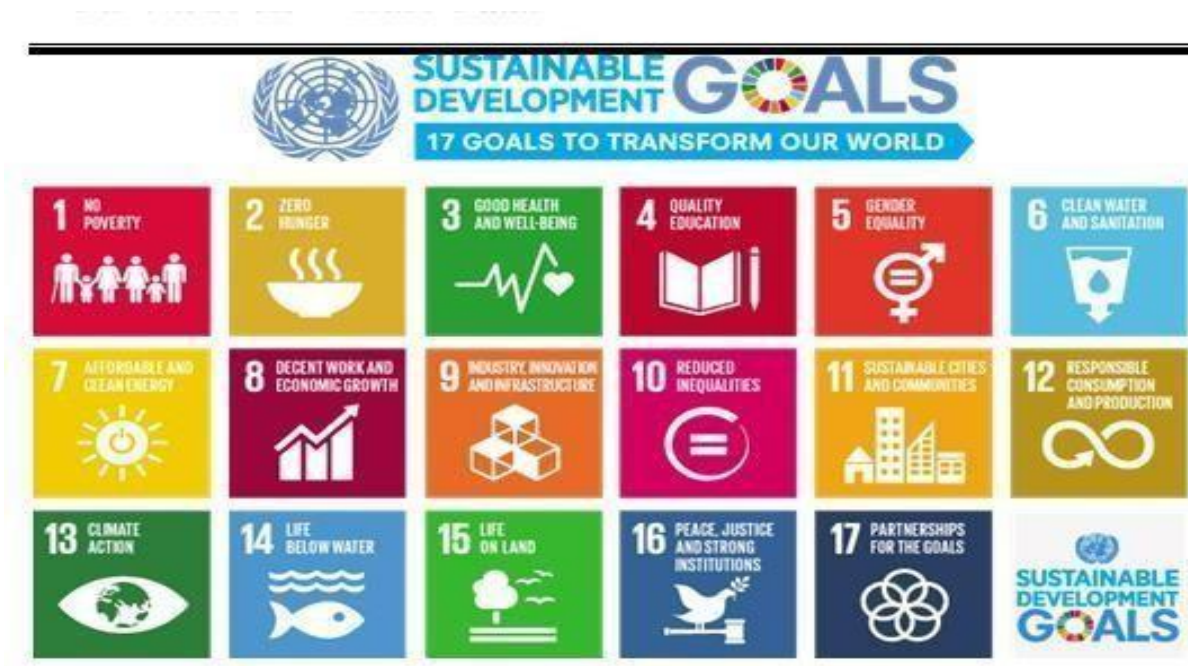


### 3. Similarity Index / Plagiarism Check report clearly showing the Percentage (%). No need for a page-wise explanation.

| Sharmasth_Vali_Y_REPORT_PLAGIARISM_CHECK_CCS-G34 |                                                                  |              |                |
|--------------------------------------------------|------------------------------------------------------------------|--------------|----------------|
| ORIGINALITY REPORT                               |                                                                  |              |                |
| 7%                                               | 5%                                                               | 2%           | 3%             |
| SIMILARITY INDEX                                 | INTERNET SOURCES                                                 | PUBLICATIONS | STUDENT PAPERS |
| PRIMARY SOURCES                                  |                                                                  |              |                |
| 1                                                | Submitted to Symbiosis International University<br>Student Paper | 1%           |                |
| 2                                                | www.globalscientificjournal.com<br>Internet Source               | <1%          |                |
| 3                                                | fastercapital.com<br>Internet Source                             | <1%          |                |
| 4                                                | ro.scribd.com<br>Internet Source                                 | <1%          |                |
| 5                                                | kcwasr.org<br>Internet Source                                    | <1%          |                |
| 6                                                | www.coursehero.com<br>Internet Source                            | <1%          |                |
| 7                                                | Submitted to 51347<br>Student Paper                              | <1%          |                |
| 8                                                | Submitted to George Mason University<br>Student Paper            | <1%          |                |
| 9                                                | Submitted to Manchester Metropolitan University                  | <1%          |                |

## 4. Sustainable Development Goals

### 9: Industry, Innovation, and Infrastructure



**Fig 6. Sustainable Development Goals - 9**

This project addresses the growing challenges of cybersecurity by building a "Resilient Digital Ecosystem" that enhances the protection of critical network infrastructure. By leveraging advanced real-time intrusion detection and innovative hybrid detection mechanisms, the system promotes the development of secure, scalable, and efficient networks. It directly aligns with SDG-9's goal of fostering innovation and building resilient infrastructure to sustain economic growth and digital transformation. The NIDES ensures the stability of modern industries and empowers organizations with tools to combat emerging cyber threats, ultimately contributing to a more secure and technologically advanced future.